Institut für Werkzeugmaschinen und Betriebswissenschaften (*iwb*)
TUM School of Engineering and Design
Technische Universität München

Entwicklung und Erprobung eines Reinforcement-Learning-Agents zur automatisierten Parametrierung von Roboterbewegungen in der Simulation

Development and Evaluation of a Reinforcement Learning Agent for the Automated Parameter Tuning of Robot Motions in Simulation

Semesterarbeit

an der TUM School of Engineering and Design der Technischen Universität München

Geprüft von Prof. Dr. Rüdiger Daub
Lehrstuhl für Produktionstechnik und Energiespeichersysteme

Eingereicht von Brenzinger Luca
Finkenweg 47
85667 Oberpframmern

Eingereicht am 20. April 2026 in Garching

Institut für Werkzeugmaschinen und Betriebswissenschaften (*iwb*)
TUM School of Engineering and Design
Technische Universität München

Aufgabenstellung

Deutscher Titel der Semesterarbeit:

Entwicklung und Erprobung eines Reinforcement-Learning-Agents zur automatisierten Parametrierung von Roboterbewegungen in der Simulation

English Title of the Semester's Thesis:

Development and Evaluation of a Reinforcement Learning Agent for the Automated Parameter Tuning of Robot Motions in Simulation

Verfasst von:	B. Sc. Luca Brenzinger
Matrikelnummer:	03796569
Studiengang:	Mechatronik, Robotik und Biomechanik
TUM School:	TUM School of Engineering and Design
Betreut von:	M. Sc. Sebastian Kurscheid
Zeitraum:	von 15.11.2025 bis 15.05.2026

Ausgangssituation
In der industriellen Produktion werden Roboter zunehmend eingesetzt, um repetitive und monotone Prozesse zu automatisieren. Die Einrichtung und Feinabstimmung der Roboterbewegungen erfordert jedoch weiterhin menschliches Fachwissen, da Programmierende Bewegungen intuitiv und iterativ anpassen, etwa indem sie bestimmte Trajektorien verlangsamen oder beschleunigen, um Energieeffizienz, Taktzeit und Prozesssicherheit zu gewährleisten. Diese manuelle Parametrierung führt zu einem hohen zeitlichen Aufwand und erschwert die Übertragbarkeit auf neue Produkt- oder Prozessvarianten. Um die Flexibilität und Effizienz in der Fertigung zu erhöhen, ist daher ein Ansatz erforderlich, der die Ableitung geeigneter Dynamikparameter automatisiert und deren Anpassung an unterschiedliche Prozessbedingungen ermöglicht.

Zielsetzung
Ziel der Arbeit ist es, den menschlichen Programmierer von der zeitaufwendigen Parametrierung zu entlasten und die Inbetriebnahme neuer Roboteranwendungen zu beschleunigen. Hierfür werden zunächst die relevanten Parameter bestimmt und klassifiziert. Dann soll ein lernbasierter Agent entwickelt werden, der die Ableitung dieser Parameter aus statischen Vorgaben automatisiert. Unter Verwendung von Prozesseingangsgrößen wie Bauteilgewicht oder Taktzeit soll der Agent die Steuerungsparameter so bestimmen, dass die festgelegten Prozessanforderungen erfüllt werden.

Vorgehensweise und Arbeitsmethodik

- Recherche und Aufarbeitung des aktuellen Standes der Technik,

- Definition des Problems und der Bewertungskriterien
- Modellierung des Entscheidungsprozesses zur Parameterableitung
- Trainingsdurchführung und Anpassung von Hyperparametern,
- Demonstration und Ergebnisbewertung
- Kritische Bewertung und Dokumentation der Ergebnisse

Vereinbarung
Durch die Betreuung von B. Sc. Luca Brenzinger durch M. Sc. Sebastian Kurscheid fließt geistiges Eigentum des *iwb* in diese Arbeit ein. Eine Veröffentlichung der Arbeit oder eine Weitergabe an Dritte bedarf der Genehmigung durch den Lehrstuhlinhaber. Der Archivierung der Arbeit in der *iwb*-eigenen und nur für Mitarbeitende des *iwb* zugänglichen Bibliothek als Bestand und in der digitalen Studienarbeitsdatenbank des *iwb* als PDF-Dokument stimme ich zu.

Garching, den 20.04.2026



Prof. Dr. Rüdiger Daub B. Sc. Luca Brenzinger

Inhaltsverzeichnis

Abbildungsverzeichnis **vii**

Tabellenverzeichnis **ix**

Formelverzeichnis **xi**

Abkürzungsverzeichnis **xiii**

1 Einleitung **1**

2 Stand der Technik **3**
2.1 Deterministische Optimierungsverfahren . . . 3
2.2 Metaheuristische Verfahren . . . 4
2.3 Bewertung deterministischer und metaheuristischer Verfahren . . . 4
2.4 Reinforcement Learning-basierte Optimierungsverfahren . . . 5
2.4.1 Prozessoptimierung durch Reinforcement Learning . . . 5
2.4.2 Parameteroptimierung durch Reinforcement Learning . . . 6

3 Ableitung des Forschungsbedarfs **7**
3.1 Nachteilige Kriterien deterministischer und metaheuristischer Optimierungsverfahren 7
3.2 Nachteilige Kriterien aktueller lernbasierter Verfahren . . . 7
3.3 Forschungslücke und Zielsetzung . . . 8

4 Anforderungsanalyse **9**
4.1 Rahmenbedingungen und Zielsetzung . . . 9
4.2 Anwendungsvariabilität . . . 10
4.3 Systemanforderungen . . . 10
4.3.1 Funktionalität . . . 11
4.3.2 Sicherheit . . . 12
4.3.3 Optimalität . . . 12
4.3.4 Adaptivität bei Variation . . . 12
4.4 Evaluierungskriterien . . . 13

5 Konzept **15**
5.1 Auswahl des Reinforcement Learning Algorithmus . . . 15
5.2 Systemarchitektur . . . 16
5.2.1 Systemaufbau . . . 16
5.2.2 Datenfluss . . . 17
5.2.3 Simulation der Bauteilmasse . . . 17
5.2.4 Bauteilschlupf-Berechnung . . . 18
5.3 Modellierung als Markov Decision Process . . . 19
5.3.1 Zustandsraum . . . 20

5.3.2 Aktionsraum 20
5.3.3 *Reward*-Funktion 21
5.4 Bewegungsbibliothek 21

6 Implementierung **23**
6.1 Simulationsumgebung 23
6.2 Gymnasium-Umgebung 24
6.2.1 Bewegungsbibliothek 24
6.2.2 *Reset*-Mechanismus 24
6.2.3 *Step*-Funktion 25
6.3 Referenzmodell und Verbesserungsberechnung 26
6.4 Trainingskonfiguration 27
6.5 Einbindung in ein modulares Ausführungssystem 28
6.5.1 *Primitive* und *Composite Skills* 28
6.5.2 Einbindung des Modells 29

7 Evaluation **31**
7.1 Versuchsaufbau 31
7.1.1 Generierung unbekannter Bewegungen 31
7.1.2 Evaluierte Metriken 32
7.2 Sicherheitsevaluation 33
7.3 Optimalitätsevaluation 35
7.3.1 PTP-Bewegungen bei unbeladenem Zustand 35
7.3.2 PTP-Bewegungen bei beladenem Zustand 36
7.3.3 LIN-Bewegungen im unbeladenen Zustand 38
7.3.4 LIN-Bewegungen im beladenen Zustand 39
7.3.5 Bewertung 40
7.4 Anforderungserfüllung 41
7.5 Ausblick 42

Literaturverzeichnis **45**

Erklärung zum Einsatz von KI-basierten Tools **47**

Abbildungsverzeichnis

5.1 **Systemarchitektur:** Zusammenspiel der Komponenten während des Trainings. 17
5.2 **Virtuelle Masse am TCP:** Zeigt die Position der virtuellen Masse in *Isaac Sim*. 18

7.1 **Erfolgsrate SAC-Modell vs. Referenzmodell:** Vergleicht die Erfolgsrate bei $a_{\text{max,zul}} = 13.0\,\text{m/s}^2$ 33
7.2 **Verteilung der gewählten Dynamikparameter im Aktionsraum bei $a_{\text{max,zul}} = 13.0\,\text{m/s}^2$.** LIN-Bewegungen (blau) werden konsequent mit niedrigen Beschleunigungsskalierungen ausgeführt, PTP-Bewegungen (orange) aggressiver. Stern: Referenzmodell. 34
7.3 **Durchschnittliche Taktzeit- und Energieverbesserung der PTP-Bewegungen im unbeladenen Zustand:** Die Abbildung zeigt die durchschnittliche Verbesserung bei verschiedenen Gewichtungsfaktoren (ew) und verschiedenen Variationsradien (r). 35
7.4 **Durchschnittliche Taktzeit- und Energieverbesserung der PTP-Bewegungen im beladenen Zu- stand:** Die Abbildung zeigt den Einfluss von $a_{\text{max,zul}}$ und `energy_weight` (ew) auf Taktzeit- und Energieverbesserung gegenüber dem Referenzmodell für PTP-Bewegungen im beladenen Zustand bei einem Variationsradius von $r = 10\,\text{cm}$. 37
7.5 **Durchschnittliche Taktzeit- und Energieverbesserung gegenüber dem Referenzmodell für LIN-Bewegungen im unbeladenen Zustand:** Die Abbildung zeigt die durchschnittliche Verbesserung bei verschiedenen Gewichtungsfaktoren (ew) und Variationsradien (r). 38
7.6 **Durchschnittliche Taktzeit- und Energieverbesserung der LIN-Bewegungen im beladenen Zustand:** Die Abbildung zeigt den Einfluss von $a_{\text{max,zul}}$ und `energy_weight` (ew) auf Taktzeit- und Energieverbesserung gegenüber dem Referenzmodell für LIN-Bewegungen im beladenen Zustand bei einem Variationsradius von $r = 10\,\text{cm}$. 40

Tabellenverzeichnis

4.1 Anwendungsvariablen des Systems, unterteilt in bauteilbezogene (V1) und prozessbezogene (V2) Parameter, die die Variation zwischen Aufgaben beschreiben. 10
4.2 Systemanforderungen A1–A4 gegliedert nach Funktionalität, Sicherheit, Optimalität und Adaptivität. 11
4.3 Messbare Evaluierungskriterien zur Verifikation der Systemanforderungen A1–A4 mit zugeordneten Erfolgsschwellen. 13

5.1 Beobachtungsvektor des RL-Agenten 20

6.1 Vergleich der *MoveIt* Bewegungsplaner 23
6.2 Hyperparameter der SAC-Trainingskonfiguration 27
6.3 *Primitive Skills*, ihre Parameter und Funktionen 28

7.1 Übersicht der Anforderungserfüllung 41

Formelverzeichnis

5.1 Coulomb'sche Haftbedingung 18
5.2 Maximal zulässige TCP-Beschleunigung 18
5.3 Beschleunigungsbetrag am TCP 19
5.4 Schlupfbedingung am TCP 19
5.5 Aktionsraum des RL-Agenten 20
6.1 Maximale Bauteilmasse an der statischen Haftgrenze 25
6.2 Reward-Funktion bei Bauteilschlupf 25
6.3 Dreistufige Reward-Funktion des RL-Agenten 26
6.4 Lineare Interpolation der Referenz-Zykluszeit 26
6.5 Lineare Interpolation des Referenz-Energieverbrauchs 26
6.6 Relative Verbesserung gegenüber dem Referenzmodell 27

Abkürzungsverzeichnis

DQN	*Deep Q-Network*
IMU	*Inertial Measurement Unit*
MDP	*Markov Decision Process*
PPO	*Proximal Policy Optimization*
RL	*Reinforcement Learning*
ROS2	*Robot Operating System 2*
SAC	*Soft Actor-Critic*
TCP	*Tool Center Point*

Kapitel 1 Einleitung

In der industriellen Produktion werden Roboter zunehmend eingesetzt, um repetitive Prozesse zu automatisieren. Die Einrichtung und Feinabstimmung der Roboterbewegungen erfordert jedoch weiterhin menschliches Fachwissen, da Programmierende Bewegungen iterativ anpassen, um Energieeffizienz, Taktzeit und Prozesssicherheit zu gewährleisten. Diese manuelle Parametrierung führt zu einem hohen zeitlichen Aufwand und erschwert die Übertragbarkeit auf neue Produkt- oder Prozessvarianten. Bestehende Ansätze wie deterministische und metaheuristische Optimierungsverfahren lösen dieses Problem nur unzureichend, da sie einen optimalen Parametersatz stets für einen einzigen, exakt definierten Anwendungsfall ermitteln (Kober et al., 2013; Talbi, 2009). Jede Änderung der Rahmenbedingungen, etwa eine Variation des Bauteils oder eine Anpassung des Optimierungsziels, erfordert eine vollständige Neu-Optimierung und führt zu messbaren Stillstandszeiten. Eine flexible, lernbasierte Alternative hingegen ermöglicht die automatisierte Ableitung geeigneter Parameter direkt aus statischen Prozessvorgaben und passt sich ohne erneute Optimierung an veränderte Bedingungen an (Lobbezoo et al., 2021). Ziel dieser Arbeit ist es, einen solchen lernbasierten Agenten auf Basis von *Reinforcement Learning* zu entwickeln, der Dynamikparameter für Roboterbewegungen automatisiert bestimmt und dabei Taktzeit, Energieverbrauch sowie Prozesssicherheit in Abhängigkeit konfigurierbarer Prozessanforderungen optimiert.

Kapitel 2 Stand der Technik

Im folgenden Kapitel wird der aktuelle Stand der Technik zur automatisierten Bestimmung von Dynamikparametern in der industriellen Robotik dargestellt. Der Fokus liegt auf Verfahren, die darauf abzielen, Roboterbewegungen optimal zu gestalten, ohne dass eine manuelle Parametrierung durch Fachpersonal erforderlich ist. Zu diesem Zweck werden zunächst deterministische und metaheuristische Optimierungsverfahren vorgestellt, die in der Robotik traditionell zur Bestimmung von Steuerungs- und Bewegungsparametern eingesetzt werden. Anschließend werden lernbasierte Ansätze betrachtet, die insbesondere in den letzten Jahren zunehmend zur Prozess- und Parameteroptimierung in der Robotik, Anwendung fanden.

2.1 Deterministische Optimierungsverfahren

Die Parametrierung von Robotersystemen kann grundsätzlich als Optimierungsaufgabe formuliert werden, wie zum Beispiel in Gasparetto und Zanotto (2007). Somit können deterministische Optimierungsverfahren zur automatischen Ermittlung von Roboterparametern in Betracht gezogen werden. Dabei wird das Ziel verfolgt, für definierte Rahmenbedingungen wie Bauteilmasse, Arbeitsraum oder Zykluszeitvorgaben einen Satz von Steuerungsparametern zu bestimmen, welcher die Systemleistung hinsichtlich vorab festgelegter Kriterien optimiert. Als Optimierungskriterien kommen unterschiedliche Zielgrößen zur Anwendung. Dazu zählt beispielsweise die Minimierung der Zykluszeit zur Steigerung der Produktivität (Bobrow et al., 1985). Ein weiteres mögliches Kriterium ist die Reduktion der Energieaufnahme zur Erhöhung der Energieeffizienz und zur Senkung der Betriebskosten (Luo et al., 2015). Darüber hinaus ist es möglich, mehrere dieser Zielkriterien zu kombinieren, wie in Gasparetto und Zanotto (2008) untersucht wurde.

Deterministische Optimierungsverfahren basieren auf einer exakten mathematischen Beschreibung des zugrunde liegenden Prozesses. Ziel ist es, durch die Definition einer Kosten- oder Zielfunktion und unter Berücksichtigung physikalischer Nebenbedingungen eine optimale Lösung zu berechnen (Nocedal & Wright, 2006). In der Robotik werden solche Verfahren häufig eingesetzt, um Steuerungs- und Bewegungsparameter eines Roboters zu ermitteln, wie etwa Reglerverstärkungen, maximale Geschwindigkeit oder Beschleunigungsgrenzen. Ein Beispiel liefert die Arbeit von Hovgard et al. (2021), die mittels eines gemischt-ganzzahligen nichtlinearen Optimierungsansatzes und deterministischem *Solver* kinematische Parameter bestimmen. Dieser Ansatz berechnet maximale Geschwindigkeiten und Beschleunigungen, die physikalisch konsistent sind und den Nebenbedingungen der Robotersteuerung entsprechen, was zu einer effizienten und stabilen Bewegungsplanung führt. Ein weiteres Verfahren von Bobrow et al. (1985) zielt auf die zeitoptimale Bahnplanung ab. Durch die Bestimmung optimaler Schaltpunkte für maximale Beschleunigung und Verzögerung entlang einer Trajektorie wird die Taktzeit unter Einhaltung der Dynamikgrenzen minimiert. Dies ermöglicht eine gezielte Parametrierung für eine zeitlich optimale und physikalisch konsistente Bewegungsausführung. Die genannten Arbeiten erfordern jedoch ein präzises mathematisches Systemmodell und sind mit hohem Rechenaufwand verbunden.

2.2 Metaheuristische Verfahren

Im Gegensatz zu deterministischen Ansätzen basieren metaheuristische Verfahren auf stochastischen Suchstrategien. Da sie keine exakte mathematische Modellierung des Systems voraussetzen, bieten sie eine flexible Alternative, die sich besonders zur Anpassung an komplexe oder unvollständig bekannte Systeme eignet. Inspiriert von natürlichen Prozessen wie der Evolution bei genetischen Algorithmen oder dem Schwarmverhalten im Fall der Partikelschwarmoptimierung durchsuchen diese stochastischen Verfahren den Lösungsraum iterativ, um optimale Lösungen zu finden. Ihr entscheidender Vorteil liegt in der Fähigkeit, komplexe oder unvollständig definierte Probleme zu behandeln, indem sie lediglich eine Bewertungs- bzw. Fitnessfunktion für vorgeschlagene Lösungen benötigen. Ein Anwendungsfeld ist die Optimierung von Reglerparametern. In Sheng und Li (2018) wird beispielsweise ein genetischer Algorithmus eingesetzt, um die PID-Parameter eines Parallelroboters zu optimieren und so die Regelgüte sowie die Robustheit gegenüber Störeinflüssen zu steigern. Auch schwarmintelligenzbasierte Algorithmen finden breite Anwendung. So wird in Akbari et al. (2023) die Partikelschwarmoptimierung genutzt, um Bewegungsparameter wie Geschwindigkeit und Beschleunigung energieoptimal auszulegen, was die Effizienz und Langlebigkeit des Roboters erhöht. In Azeez et al. (2023) wird ein Artificial Bee Colony Algorithmus zur Optimierung von PID-Reglern verwendet, wobei eine neuartige, auf Lyapunov-Funktionen basierende Zielfunktion zu einer besonders robusten Steuerung gegenüber Störungen und variablen Nutzlasten führt.

Trotz ihrer aufgezeigten Potenziale sind metaheuristische Verfahren durch ihre im Kern statische Natur wesentlich eingeschränkt. Da die ermittelten Parameter meist für einen einzigen, exakt definierten Systemzustand optimiert sind, erfordert jede Änderung der Rahmenbedingungen, wie etwa eine Variation der Bauteilmasse, eine vollständige Neu-Optimierung. Dieser hohe Rekonfigurationsaufwand ist rechenintensiv und verhindert eine adaptive Anpassung an wechselnde Prozessbedingungen.

2.3 Bewertung deterministischer und metaheuristischer Verfahren

Die wesentliche Einschränkung deterministischer Verfahren ist ihre Abhängigkeit von einem exakten Systemmodell (Kober et al., 2013). Metaheuristische Ansätze umgehen diese Hürde, da sie modellfrei arbeiten und somit auch komplexe oder unvollständig bekannte Systeme optimieren können (Talbi, 2009). Die Schwäche beider Ansätze liegt jedoch in ihrer mangelnden Adaptivität. Sie fungieren als statische Optimierer für einen spezifischen Systemzustand. Diese fehlende Adaptivität ist insbesondere für die automatisierte Parameterbestimmung problematisch, da Robotersysteme in der Praxis verschiedene Bewegungen und Szenarien bewältigen müssen, die mit variierenden Parametern einhergehen. Statische Optimierungsverfahren, die für einen festen Systemzustand ausgelegt sind, können diese Anforderung nicht erfüllen (Kober et al., 2013). Diese Grenzen klassischer Verfahren machen deutlich, dass für flexible Robotersysteme, neue Ansätze erforderlich sind, die ohne exakte Modellkenntnis auskommen und eine selbstständige Anpassung an wechselnde Bedingungen ermöglichen.

2.4 Reinforcement Learning-basierte Optimierungsverfahren

Die in den vorangegangenen Abschnitten diskutierten Grenzen deterministischer und metaheuristischer Verfahren haben die Entwicklung hin zu datengetriebenen Ansätzen vorangetrieben. Innerhalb des maschinellen Lernens wird dabei zwischen *Supervised Learning*, *Unsupervised Learning* und *Reinforcement Learning* (RL) unterschieden. Für die Problemstellung der automatisierten Parameterfindung erweisen sich die ersten beiden Ansätze jedoch als ungeeignet. Das *Supervised Learning* basiert auf dem Training mit anotierten Daten, also Eingaben mit bereits bekannten, korrekten Lösungen. Da das Ziel der Automatisierung jedoch gerade darin besteht, diese optimalen Parameter erst zu ermitteln, liegen solche Label initial nicht vor. Eine manuelle Erstellung der Label für jede mögliche Prozess- und Bauteilvariante wäre mit einem unverhältnismäßig hohen Aufwand verbunden und würde den Nutzen einer automatisierten Lösung hinfällig machen. Das *Unsupervised Learning* scheidet aus, weil es vor allem darauf abzielt, Zusammenhänge in Daten zu erkennen, jedoch keine gezielte Optimierung einer klar definierten Leistungskennzahl verfolgt.

Folglich konzentriert sich der aktuelle Stand der Technik auf das *Reinforcement Learning*. Wie von Lobbezoo et al. (2021) in ihrem Survey zu *Pick-and-Place-Anwendungen* dargelegt wird, ermöglicht RL einem Roboteragenten, Aufgaben ohne explizite Programmierung durch Versuch und Irrtum zu erlernen. Der Agent interagiert mit seiner Umgebung und passt seine Strategie basierend auf der *Reward-Function* an, um ein definiertes Ziel zu erreichen. Viele dieser Arbeiten konzentrieren sich jedoch auf das Erlernen oder Optimieren der Bewegungssequenzen selbst, also auf die direkte Steuerung der Gelenkwinkel oder des Endeffektors (Lobbezoo et al., 2021). In dieser Arbeit steht jedoch nicht das Erlernen der Bewegung selbst im Mittelpunkt, sondern die automatisierte Ableitung und Optimierung von Dynamikparametern. Die folgenden Abschnitte beleuchten daher, wie RL zur Optimierung von Gesamtprozessen und zur gezielten Bestimmung von Parametern eingesetzt wird.

2.4.1 Prozessoptimierung durch Reinforcement Learning

Über das Erlernen von Bewegungsabläufen hinaus wird *Reinforcement Learning* zunehmend zur Optimierung von Prozessparametern in industriellen Anwendungen eingesetzt. Ziel ist es, einen optimalen Kompromiss zwischen konkurrierenden Zielen zu finden. Ein anschauliches Beispiel hierfür liefert die Arbeit von Burggräf et al. (2022), die RL zur Optimierung der Prozesszeit in einer automatisierten Montagezelle einsetzen. In diesem Anwendungsfall wird ein RL-Agent mittels des *Q-Learning-Algorithmus* trainiert, um die Geschwindigkeiten von Förderbändern in einer physischen Industrie-4.0-Demonstrationszelle zu steuern. Der Zustand des Systems wird dabei durch die Kombination der Geschwindigkeiten von zwei Förderbändern definiert, während die Aktionen des Agenten aus einem Satz diskreter Befehle zur Erhöhung oder Verringerung der jeweiligen Bandgeschwindigkeiten bestehen. Ein ähnlicher Ansatz zur Prozessoptimierung findet sich bei Paranjape et al. (2025), die RL zur multikriteriellen Online-Optimierung von Fertigungsparametern einsetzen, um mehrere konkurrierende Qualitätsmerkmale gleichzeitig auszubalancieren. Diese Beispiele verdeutlichen, dass RL in der Lage ist, komplexe Abhängigkeiten innerhalb eines Prozesses zu erlernen und Stellgrößen so anzupassen, dass eine übergeordnete, multikriterielle Zielfunktion optimiert wird.

2.4.2 Parameteroptimierung durch Reinforcement Learning

Aufbauend auf der Optimierung von Prozessen lässt sich RL auch gezielt zur Bestimmung von Dynamikparametern einsetzen, was den Kern der vorliegenden Arbeit darstellt. Anstatt Prozessvariablen wie Bandgeschwindigkeiten zu steuern, lernt der Agent hier, spezifische Parameter für die Bewegungssteuerung eines Roboters festzulegen. Anwendungsbeispiele reichen von Aufgaben wie der Optimierung von Schleifparametern in Pan et al. (2025), bis hin zur Anpassung kinematischer Parameter. Von besonderer Relevanz ist hier die Arbeit von Xu und Wang (2023), die ein *Deep Reinforcement Learning Framework* für das *visual Servoing* von Robotern entwickeln. Ziel ist die Automatisierung der zeitvarianten Konfiguration von Kinematikparametern. Konkret wird ein *Deep Reinforcement Learning Agent* trainiert, um Parameter der Vorwärtskinematik zur Maximierung der Bewegungsgeschwindigkeit und gleichzeitig Parameter der inversen Kinematik zur Sicherstellung der Bewegungsglattheit zu optimieren. Ein wesentliches Merkmal der vorgestellten Methode ist jedoch, dass die Parameteranpassung reaktiv auf Basis von kontinuierlichem, visuellem *Feedback* während der Bewegungsausführung erfolgt. Der Agent beobachtet den Systemzustand in Echtzeit über Kameras und passt die Kinematikparameter dynamisch an.

Kapitel 3 Ableitung des Forschungsbedarfs

Die Analyse des Stands der Technik zeigt eine klare Entwicklung von modellbasierten, statischen Optimierungsverfahren hin zu datengetriebenen, lernbasierten Ansätzen, welche zur automatisierten Parametrierung, von Robotersystemen und Prozessen, genutzt werden (Chen et al., 2025). Sowohl die klassischen als auch die aktuellen lernbasierten Methoden weisen jedoch im Hinblick auf eine flexible und prädiktive Automatisierung von Dynamikparametern spezifische, messbare Nachteile auf, die den Bedarf für einen neuen Ansatz begründen.

3.1 Nachteilige Kriterien deterministischer und metaheuristischer Optimierungsverfahren

Die primäre Schwäche der deterministischen und metaheuristischen Verfahren liegt in ihrer statischen Natur. Sie ermitteln einen optimalen Parametersatz für einen einzigen, exakt definierten Anwendungsfall. Dies führt zu Nachteilen in einem modernen, flexiblen Produktionsumfeld:

1. **Hoher Rekonfigurationsaufwand:** Jede Änderung der Rahmenbedingungen, wie eine Variation des Bauteils beziehungsweise dessen Masse oder eine Anpassung der Taktzeit, erfordert eine vollständige Neu-Optimierung. Dieser Prozess ist rechenintensiv und führt zu messbaren Stillstandszeiten der Anlage, was die Gesamtanlageneffektivität direkt reduziert und den Aufwand der Parameterauswahl zusätzlich erschwert.
2. **Quantifizierbarer Modellierungsaufwand:** Insbesondere deterministische Verfahren erfordern ein hochpräzises mathematisches Modell. Der Aufwand für die Erstellung, Validierung und Pflege dieses Modells ist in Ingenieurstunden quantifizierbar und stellt eine erhebliche Kosten- und Zeitbarriere dar. Modellungenauigkeiten führen zudem zu einer messbaren Abweichung zwischen dem berechneten Optimum und der real erreichbaren Systemleistung.

3.2 Nachteilige Kriterien aktueller lernbasierter Verfahren

Lernbasierte Ansätze, insbesondere das *Reinforcement Learning*, überwinden die Hürde der Modellabhängigkeit und bieten Potenzial für Adaptivität. Die im Stand der Technik identifizierten Anwendungen weisen jedoch eine Fokusverschiebung auf, die für die hier adressierte Problemstellung nachteilig ist:

1. **Fokus auf reaktive Online-Anpassung statt prädiktiver Parametrierung:** Ansätze wie in Xu und Wang (2023) beschrieben, optimieren Parameter während der Bewegungsausführung.

Dies erfordert eine kontinuierliche, latenzarme Verarbeitung von Sensordaten und erzeugt einen permanenten Rechenaufwand auf dem Steuerungssystem. Der wesentliche Nachteil dieser Verfahren besteht darin, dass sie keine vorgelagerte Parametrierung der Bewegungsplanung ermöglichen. Zwar können sie eine suboptimale Startkonfiguration nachträglich korrigieren, jedoch nicht die initial optimale Konfiguration auf Grundlage der vorhandenen Prozessvorgaben bestimmen.

2. **Fokus auf ungeeignete Parameterdomänen:** Eine grundlegende Schwäche bestehender RL-Ansätze besteht darin, dass sie auf einer ungeeigneten Abstraktionsebene operieren und dabei jene zu automatisierenden Parameter außer Acht lassen, die bislang manuell, iterativ und mit Aufwand abgeleitet werden müssen. Die Literatur konzentriert sich primär auf zwei Extreme:

   - **Direkte Trajektorien-Optimierung:** Ein Großteil der Forschung nutzt RL, um die Bewegungsbahn selbst, also die Sequenz von Gelenkwinkeln oder Endeffektor-Posen, zu erlernen (Lobbezoo et al., 2021). Dies adressiert somit die Frage, welche Bewegung ausgeführt werden soll, jedoch nicht, wie eine vorgegebene Bewegung optimal parametriert werden kann. Folglich stellen diese Verfahren keine geeignete Grundlage für die automatisierte Ableitung der Dynamikparameter vordefinierter Bewegungsabläufe dar.
   - **Übergeordnete Prozess-Optimierung:** Ansätze wie von Burggräf et al. (2022) optimieren übergeordnete Prozessgrößen (z. B. Bandgeschwindigkeiten) und behandeln den Roboter als eine Blackbox mit fixen, unveränderlichen Bewegungseigenschaften. Damit bleibt nicht nur ein erhebliches Optimierungspotenzial ungenutzt, sondern die robotereigenen Dynamikparameter müssen weiterhin manuell beziehungsweise eigenständig ermittelt werden.

   Beide Ansätze verfehlen die entscheidende, mittlere Ebene: die der Bewegungs- und Dynamikparameter wie zum Beispiel der maximalen achsspezifischen Geschwindigkeit oder Beschleunigung. In der industriellen Praxis werden diese Parameter manuell justiert, um einen Kompromiss im Rahmen multiobjektiver Prozesse zu erzielen. Indem bestehende RL-Methoden entweder die gesamte Bewegung neu lernen oder den Roboter als starre Einheit betrachten, adressieren sie nicht den eigentlichen Freiheitsgrad, der für eine flexible und effiziente Produktion optimiert und automatisiert werden muss. Dabei ist zu berücksichtigen, dass diese Lücke nicht auf grundlegende methodische Grenzen beruht. Wie die Analyse in Abschnitt 2.4 gezeigt hat, ist RL bereits nachweislich in der Lage, sowohl übergeordnete Prozessabläufe zu optimieren als auch spezifische technische Parameter präzise zu bestimmen. Da die Methode somit prinzipiell beide erforderlichen Kompetenzen vereint, liegt der Schluss nahe, dass sie sich auch für die hier adressierte Ebene eignet.

3.3 Forschungslücke und Zielsetzung

Zusammenfassend lässt sich feststellen, dass im Stand der Technik eine Lücke existiert, da eine Methode fehlt, die auf Basis von Eingangsparametern prädiktiv die optimalen Bewegungs- und Dynamikparameter bestimmt, um so die manuelle Parameterauswahl zu automatisieren. Ein solches System würde die Adaptivität von RL-Ansätzen nutzen, ohne den Nachteil des reaktiven Online-Rechenaufwands und der Abhängigkeit von kontinuierlichem Sensorfeedback in Kauf nehmen zu müssen. Es würde die statische Natur der deterministischen und metaheuristischen Verfahren überwinden und eine schnelle, automatisierte Rekonfiguration ermöglichen.

Kapitel 4 Anforderungsanalyse

Auf Basis der in Kapitel 2 identifizierten Forschungslücke werden in diesem Kapitel die konkreten Anforderungen an das zu entwickelnde System zur automatisierten Bestimmung von Dynamikparametern abgeleitet. Um den Lösungsraum der Arbeit klar einzugrenzen, werden zunächst die übergeordneten Rahmenbedingungen und die genaue Zielsetzung des Systems definiert. Daran anschließend werden die anwendungsspezifischen Variablen beschrieben, die von Aufgabe zu Aufgabe variieren und auf die das System flexibel reagieren muss. Die daraus abgeleiteten Systemanforderungen strukturieren sich in vier Kategorien: Funktionalität, Sicherheit, Optimalität und Adaptivität. Dabei stellen die Anforderungen der Funktionalität und Sicherheit notwendige Bedingungen für eine gültige Bewegungsausführung dar und werden als harte Randbedingungen betrachtet. Die weitere Optimierung der Bewegungen erfolgt erst innerhalb des durch diese Bedingungen definierten Lösungsraums. Ergänzend werden die anwendungsspezifischen Variablen beschrieben, die von Aufgabe zu Aufgabe variieren und auf die das System flexibel reagieren muss, sowie die messbaren Kriterien zur Verifikation der Anforderungserfüllung.

4.1 Rahmenbedingungen und Zielsetzung

Das entwickelte System adressiert ausschließlich die vorgelagerte, prädiktive Bestimmung von Dynamikparametern für vordefinierte Bewegungsabläufe. Ziel ist es, die Zykluszeit unter Einhaltung aller Sicherheitsanforderungen zu minimieren, wobei eine anwendungsspezifische Gewichtung zugunsten der Energieeffizienz möglich ist. Die folgenden Aspekte definieren den Rahmen der Arbeit und grenzen den Ansatz explizit von verwandten Problemstellungen ab.

Keine Optimierung der Bewegungsbahn: Das System optimiert ausschließlich die Dynamikparameter `max_vel` und `max_acc` für vorgegebene Bewegungsabläufe. Die Bewegungsbahn selbst wird nicht durch den Agenten bestimmt oder verändert. Der Fokus liegt damit auf der Automatisierung der manuellen Parametrierung bestehender Bewegungen, welche in der industriellen Praxis bislang iterativ durch Fachpersonal erfolgt und erheblichen Aufwand verursacht.

Keine reaktive Online-Anpassung: Die Parameterbestimmung erfolgt einmalig vor jeder Bewegungsausführung auf Basis statischer Eingangsgrößen. Der Agent wird vor jeder Bewegung aufgerufen und setzt die Parameter, bevor die Ausführung beginnt. Eine Anpassung während der laufenden Bewegung ist weder vorgesehen noch erforderlich. Dies grenzt den Ansatz explizit von reaktiven Verfahren wie dem in Xu und Wang (2023) beschriebenen *Visual-Servoing-Framework* ab, bei dem Parameter kontinuierlich auf Basis von Sensordaten angepasst werden.

Stationsbindung: Das System wird für eine definierte Roboterstation trainiert und evaluiert. Die Generalisierung adressiert sowohl strukturell ähnliche Bewegungen innerhalb dieser Station als auch variierende Bauteilparameter, insbesondere Masse, Reibkoeffizient und Greifkraft. Bereitstellungs- und Montageumgebungen innerhalb einer Station weisen eine hohe kinematische Ähnlichkeit auf, sodass ein stationsspezifisch trainiertes Modell eine praxisrelevante Abdeckung erreicht.

Eine Übertragung auf grundlegend andere Stationen mit abweichenden Arbeitsräumen oder Bewegungscharakteristiken ist nicht Gegenstand dieser Arbeit und würde ein Neu-Training erfordern.

Roboterbindung: Das trainierte Modell ist an einen spezifischen Roboter gebunden. Da `max_vel` und `max_acc` relative Skalierungen der roboterspezifischen Maximalwerte darstellen, haben identische Parameterwerte auf unterschiedlichen Roboterkinematiken eine abweichende physikalische Bedeutung. Zudem basiert die Schlupf-Berechnung auf kinematikabhängigen Endeffektor-Beschleunigungen. Eine direkte Übertragung auf andere Roboterkinematiken ist daher nicht vorgesehen und erfordert ein Neu-Training.

Robustheit gegenüber Messrauschen: Die Robustheit des Agenten gegenüber Ungenauigkeiten oder Rauschen in den Eingangsparametern ist nicht Gegenstand der Evaluation dieser Arbeit und bleibt weiterführenden Untersuchungen vorbehalten.

4.2 Anwendungsvariabilität

Das zu entwickelnde System soll in der Lage sein, für unterschiedliche Aufgaben geeignete Dynamikparameter zu bestimmen. Die Variabilität zwischen einzelnen Aufgaben ergibt sich aus zwei Kategorien, die nachfolgend beschrieben werden.

Tabelle 4.1: Anwendungsvariablen des Systems, unterteilt in bauteilbezogene (V1) und prozessbezogene (V2) Parameter, die die Variation zwischen Aufgaben beschreiben.

Kategorie	Variable	Beschreibung
V1 Bauteilbezogen	Bauteilmasse	Masse des zu handhabenden Bauteils in kg.
	Reibkoeffizient	Reibungskoeffizient zwischen Greifer und Bauteil.
	Max. Greifkraft	Maximale Kraft, die der Greifer auf das Bauteil ausüben darf.
V2 Prozessbezogen	Gewichtung der Taktzeit / Energie	Skalarer Gewichtungsfaktor zwischen Zykluszeit-Minimierung und Energieeffizienz.
	Start- und Zielpose	Start- und Endposition der Roboterbewegungen.

4.3 Systemanforderungen

Die Anforderungen an das System wurden in vier Kategorien untergliedert: Funktionalität (A1), Sicherheit (A2), Optimalität (A3) sowie Adaptivität (A4), welche in Tabelle 7.1 erläutert werden.

Tabelle 4.2: Systemanforderungen A1–A4 gegliedert nach Funktionalität, Sicherheit, Optimalität und Adaptivität.

ID	Anforderung	Beschreibung
A1.1	Prädiktive Parameterbestimmung	Der Agent bestimmt aus statischen Eingangsparametern prädiktiv die Dynamikparameter `max_acc` und `max_vel`, bevor die Bewegung ausgeführt wird.
A1.2	Parameterübergabe an die Simulation	Der Agent setzt die ermittelten Parameter in der Simulationsumgebung, sodass die Simulation der Bewegung auf deren Basis abläuft.
A2.1	Einhaltung der Robotergrenzen	Die gewählten Parameter dürfen keine Systemgrenzen des Roboters verletzen.
A2.2	Kollisionsfreiheit	Während der Bewegungsausführung darf es zu keinen Kollisionen mit der Umgebung, dem Werkstück oder dem Roboter selbst kommen.
A2.3	Schlupffreie Bauteilhandhabung	Das Bauteil darf während der Bewegungsausführung des Roboters nicht im Greifer verrutschen.
A3.1	Minimierung von Taktzeit und Energie	Die Dynamikparameter sind so zu wählen, dass die Zykluszeit unter Einhaltung von A1–A2 minimiert wird. Der anwendungsspezifische Gewichtungsfaktor (V2) erlaubt eine gezielte Verschiebung des Optimierungsziels zugunsten der Energieeffizienz.
A4.1	Generalisierung auf Stationsebene	Der trainierte Agent muss die Anforderungen A1–A3 für strukturell ähnliche, nicht im Training enthaltene Bewegungen derselben Roboterstation sowie für unbekannte Kombinationen der Variablen V1–V2 erfüllen.

4.3.1 Funktionalität

Die funktionalen Anforderungen beschreiben die Kernkompetenz des Systems. Im Gegensatz zu reaktiven Ansätzen, wie etwa dem in Xu und Wang (2023) beschriebenen *Visual-Servoing-Framework*, soll der Agent nicht während der Bewegungsausführung eingreifen, sondern die Parameter prädiktiv auf Basis statischer Eingaben bestimmen. A1.1 adressiert damit direkt die in Kapitel 3 identifizierte Forschungslücke der fehlenden vorgelagerten Parametrierung der Bewegungsplanung.

A1.2 stellt die technische Schnittstelle zwischen Agent und Simulationsumgebung sicher. Die vom Agenten gesetzten Parameter müssen von der Simulation korrekt übernommen und fehlerfrei verarbeitet werden. Dies ist eine grundlegende Voraussetzung sowohl für ein valides Training als auch für den späteren Einsatz. Nur wenn die Parameterentscheidungen des Agenten tatsächlich die Bewegungsausführung beeinflussen, kann eine sinnvolle Optimierung stattfinden. Die Erfüllung dieser Anforderung wird durch die Systemarchitektur strukturell sichergestellt und erfordert keine separate Evaluation.

4.3.2 Sicherheit

Die Sicherheitsanforderungen gewährleisten, dass die jeweilige Aufgabe grundsätzlich erfolgreich ausgeführt werden kann. Sie haben in der Anforderungshierarchie Vorrang vor der Optimalität (A3), da eine fehlgeschlagene Aufgabenausführung in der Praxis stets mit größeren negativen Konsequenzen verbunden ist als eine suboptimale Parameterwahl.

A2.1 stellt sicher, dass die gewählten Parameter nicht zu einer mechanischen Überlastung des Roboters führen. Werden Drehmoment-, Kraft-, Beschleunigungs- oder Geschwindigkeitsgrenzen überschritten, gefährdet dies die Hardware und kann den Betrieb unterbrechen.

A2.2 fordert Kollisionsfreiheit während der gesamten Bewegungsausführung. Da der Agent die Bewegungsbahn nicht verändert, sondern nur bestehende Bewegungen parametriert, hängt die Kollisionsfreiheit nicht vom Modell ab. Sie wird durch den eingesetzten Bewegungsplaner gewährleistet und daher nicht separat evaluiert.

A2.3 sichert eine schlupffreie Bauteilhandhabung während der gesamten Bewegungsausführung. Überschreiten die durch die gewählten Parameter entstehenden Endeffektor-Beschleunigungen den zulässigen Grenzwert, kann das Bauteil im Greifer verrutschen, was die Prozesszuverlässigkeit beeinträchtigt und die Wiederholgenauigkeit der Anwendung reduziert.

4.3.3 Optimalität

A3.1 formuliert die eigentliche Optimierungsaufgabe. Das primäre Ziel des Agenten besteht darin, die Zykluszeit unter Einhaltung der Sicherheitsanforderungen A2 aktiv zu minimieren. Es geht dabei ausdrücklich nicht darum, eine vorgegebene Zieltaktzeit einzuhalten, sondern die kürzestmögliche Ausführungszeit bei gegebenen Randbedingungen zu erzielen. Über den prozessbezogenen Gewichtungsfaktor (V2) kann dieses Optimierungsziel gezielt zugunsten einer reduzierten Energieaufnahme verschoben werden, sodass der Agent je nach Konfiguration entweder maximale Geschwindigkeit oder einen energieeffizienten Betrieb anstrebt.

4.3.4 Adaptivität bei Variation

A4.1 konkretisiert die Generalisierungsfähigkeit des trainierten Modells im Kontext einer stationsbezogenen Anwendung. Das System wird mit einer repräsentativen Teilmenge häufig vorkommender Bewegungen einer Roboterstation trainiert. Die Anforderung besteht darin, dass der Agent die Anforderungen A1–A3 auch für strukturell ähnliche, jedoch nicht im Training enthaltene Bewegungen derselben Station sowie für variierende Bauteilparameter (V1) und Prozessgewichtungen (V2) erfüllt. Dieser Ansatz ist dadurch motiviert, dass Bereitstellungs- und Montageräume innerhalb einer Station eine hohe strukturelle Ähnlichkeit aufweisen. Die auftretenden Bewegungen unterscheiden sich zwar in Start- und Zielpose, bleiben jedoch in ihrer kinematischen Charakteristik vergleichbar. Die Generalisierung auf grundlegend andere Stationen oder Roboterkinematiken ist bewusst nicht Gegenstand dieser Arbeit.

4.4 Evaluierungskriterien

Zur objektiven Überprüfung der Anforderungserfüllung werden messbare Evaluierungskriterien (K) definiert, die den jeweiligen Anforderungen A1–A4 direkt zugeordnet sind.

Tabelle 4.3: Messbare Evaluierungskriterien zur Verifikation der Systemanforderungen A1–A4 mit zugeordneten Erfolgsschwellen.

Kriterium	Anforderung	Messbares Erfolgskriterium
K1.1.1	A1.1	Der Agent gibt `max_vel` und `max_acc` auf Basis der Eingangsparameter aus, bevor die Bewegungsausführung beginnt. Die Parameterübergabe ist im Ausführungslog nachvollziehbar.
K2.1.1	A2.1	Anzahl der durch *MoveIt* abgelehnten Bewegungsziele durch das Nicht-Einhalten von Robotergrenzen $= 0$ in allen Testszenarien.
K2.3.1	A2.3	Gemessene TCP-Beschleunigung $a_{\mathrm{TCP}} < a_{\mathrm{max,zul}}$ in allen Testszenarien, wobei $a_{\mathrm{max,zul}}$ aus Greifkraft, Bauteilmasse und Reibkoeffizient bestimmt wird.
K3.1.1	A3.1	Bei `energy_weight` ≤ 0.2: Zykluszeit des Agenten $\geq 10\,\%$ unter der Zykluszeit des Referenzmodells.
K3.1.2	A3.1	Bei `energy_weight` ≥ 0.8: Energieverbrauch des Agenten $\geq 10\,\%$ unter Baseline-Energieverbrauch.
K4.1.1	A4.1	Einhalten von A2 und messbare Verbesserung gemäß A3 bei 200 unbekannten Test-Bewegungen, die nicht in der Trainingsbibliothek enthalten sind.

Kapitel 5 Konzept

Basierend auf den in Kapitel 4 definierten Systemanforderungen wird im Folgenden ein Konzept zur automatisierten und prädiktiven Bestimmung von Dynamikparametern entwickelt. Wie die Analyse des Stands der Technik (vgl. Kapitel 2.4) gezeigt hat, stellt das *Reinforcement Learning* (RL) hierfür den am besten geeigneten Ansatz dar, da es eine gezielte Optimierung ermöglicht, ohne auf ein exaktes mathematisches Modell oder vorab annotierte Trainingsdaten angewiesen zu sein. Hierfür wird zunächst dargelegt, welche spezifischen Problemcharakteristika die Wahl des konkreten RL-Algorithmus motivieren. Daran anschließend werden die Systemarchitektur sowie die formale Modellierung der Aufgabe als *Markov Decision Process* (MDP) vorgestellt. Abschließend wird die zugrundeliegende Bibliothek an Trainingsbewegungen definiert.

5.1 Auswahl des Reinforcement Learning Algorithmus

Die Wahl des RL-Algorithmus ergibt sich aus drei Eigenschaften des vorliegenden Problems, der Kontinuität des Aktionsraums, dem erforderlichen Explorationsverhalten über einen heterogenen Parameterraum sowie der Dateneffizienz angesichts der hohen Anzahl an nötigen Trainingschritten.

Kontinuierlicher Aktionsraum: Die zu bestimmenden Parameter `max_vel` und `max_acc` sind kontinuierliche Größen im Intervall $[0.01,\ 1.0]$. Algorithmen die auf diskreten Aktionsräumen operieren, wie etwa *Q-Learning* oder *Deep Q-Network* (DQN) (Mnih et al., 2015), sind daher nicht direkt anwendbar und erfordern eine Diskretisierung die zwangsläufig Präzision kostet.

Explorationsverhalten: Die optimalen Parameter variieren je nach Gewichtung von Energie, Taktzeit und Bewegungstyp erheblich, bei zeitoptimaler Gewichtung liegen sie in einem anderen Bereich des Aktionsraums als bei energieoptimaler Gewichtung. Ein Algorithmus der zu früh auf eine bestimmte Parameterkombination konvergiert, läuft Gefahr relevante Bereiche des Aktionsraums nicht ausreichend zu explorieren.

Dateneffizienz: Wie Kober et al. (2013) zeigen, benötigen RL-Verfahren generell eine hohe Anzahl an Interaktionen mit der Umgebung, um eine stabile *Policy* , also ein konsistentes Entscheidungsverhalten, zu erlernen. Ein dateneffizienter Algorithmus der vergangene Erfahrungen mehrfach nutzt ist daher einem *On-Policy*-Verfahren vorzuziehen, das jeden gesammelten Datenpunkt nur einmal für ein Update verwendet.

Fazit: Der *Soft Actor-Critic* (SAC)-Algorithmus (Haarnoja et al., 2018) erfüllt diese drei Anforderungen optimal und erweist sich insbesondere im Vergleich zum ebenfalls weit verbreiteten *Proximal Policy Optimization* (PPO)-Algorithmus (Schulman et al., 2017) als vorteilhafter für den vorliegenden Anwendungsfall. Zwar sind beide Algorithmen in der Lage, direkt auf kontinuierlichen Aktionsräumen zu operieren, jedoch bietet SAC bei den Kriterien Exploration und Dateneffizienz entscheidende Vorteile.

Zur Sicherstellung des Explorationsverhaltens wird die Zielfunktion des Agenten bei SAC durch das

Prinzip des *Maximum Entropy RL* um einen zusätzlichen Entropie-Term erweitert. Dieser belohnt den Agenten, wenn er möglichst vielfältige Aktionen wählt. Dies verhindert dass der Algorithmus zu früh auf eine einzige Parameterkombination konvergiert und relevante Bereiche des Aktionsraums unentdeckt bleiben.

Der signifikanteste Vorteil gegenüber PPO liegt jedoch in der Dateneffizienz. Im Gegensatz zu PPO, welches als *On-Policy*-Verfahren jeden gesammelten Datenpunkt nur für ein einziges Modell-Update verwendet und danach verwirft, handelt es sich bei SAC um einen *Off-Policy*-Algorithmus. SAC nutzt einen *Replay Buffer*, aus dem vergangene Erfahrungen wiederholt für Modell-Updates entnommen werden. Dies erhöht die Dateneffizienz wesentlich, da jede gesammelte Erfahrung mehrfach genutzt wird. Gerade bei rechenintensiven physikalischen Simulationen reduziert dies die Anzahl der benötigten Trainingsinteraktionen mit der Umgebung und damit die Gesamttrainingszeit deutlich.

5.2 Systemarchitektur

In diesem Abschnitt wird die Systemarchitektur des entwickelten Ansatzes sowie dessen Funktionsweise beschrieben. Zunächst wird ein Überblick über die beteiligten Systemkomponenten und deren Zusammenspiel gegeben. Anschließend wird der Ablauf einer Trainingsepisode in Form des Datenflusses erläutert. Darauf aufbauend werden die zentralen physikalischen Modellierungen vorgestellt, insbesondere die Simulation variierender Bauteilmassen sowie die Berechnung des Bauteilschlupfs als sicherheitskritische Nebenbedingung.

5.2.1 Systemaufbau

Das entwickelte System besteht aus vier funktionalen Komponenten, die über definierte Schnittstellen miteinander kommunizieren. Hierzu gehören die physikbasierte Simulationsumgebung *Isaac Sim* (NVIDIA Corporation, 2023), ein *Task Metrics Node* zur Berechnung relevanter Metriken, der Bewegungsplaner *MoveIt* (PickNik Robotics, 2021) sowie ein *Gymnasium-Environment* (Towers et al., 2024). Abbildung 5.1 zeigt das Zusammenspiel dieser Komponenten im Überblick.

Die physikalische Umgebung und der Roboter werden in *Isaac Sim* simuliert. Die Simulation bildet Gelenkdynamik, Trägheit und Bauteilphysik ab und stellt dem Gesamtsystem über die ROS2-*Bridge* Gelenkzustände und Sensordaten bereit.

Das *Robot Operating System 2* (ROS2) übernimmt die zentrale Kommunikation zwischen allen Systemkomponenten. Informationen wie Gelenkzustände, Steuerkommandos und Sensordaten werden dabei über *Topics* und *Services* ausgetauscht. *MoveIt* ist als standardisiertes *Framework* direkt in ROS2 eingebunden, und übernimmt die Bewegungsplanung auf Basis der vom Agenten gesetzten Dynamikparameter. *Isaac Sim* wird über die ROS2-*Bridge* und die ROS2-*Simulation Control*-Erweiterungen mit den ROS2 Schnittstellen gekoppelt.

Der *Task Metrics Node* ist ein ROS2-Knoten, der Rohdaten aus *Isaac Sim* entgegennimmt und daraus die für den Agenten relevanten Metriken berechnet. Er abonniert die Gelenkzustände zur Berechnung des Energieverbrauchs sowie die Daten der *Inertial Measurement Unit* (IMU) zur Erfassung der Endeffektor-Beschleunigungen. Die berechneten Metriken werden nach Abschluss jeder Bewegung über einen *Service* an die Gymnasium-Umgebung übergeben, wo sie in die *Reward*-Berechnung einfließen. Die Auslagerung in einen separaten Knoten ist dadurch begründet, dass die Metrikberechnung einen kontinuierlichen Datenempfang von Gelenkzuständen und IMU-Daten erfordert, während die Gymnasium-Umgebung in jeder Episode blockiert, bis die Bewegungsausführung ab-

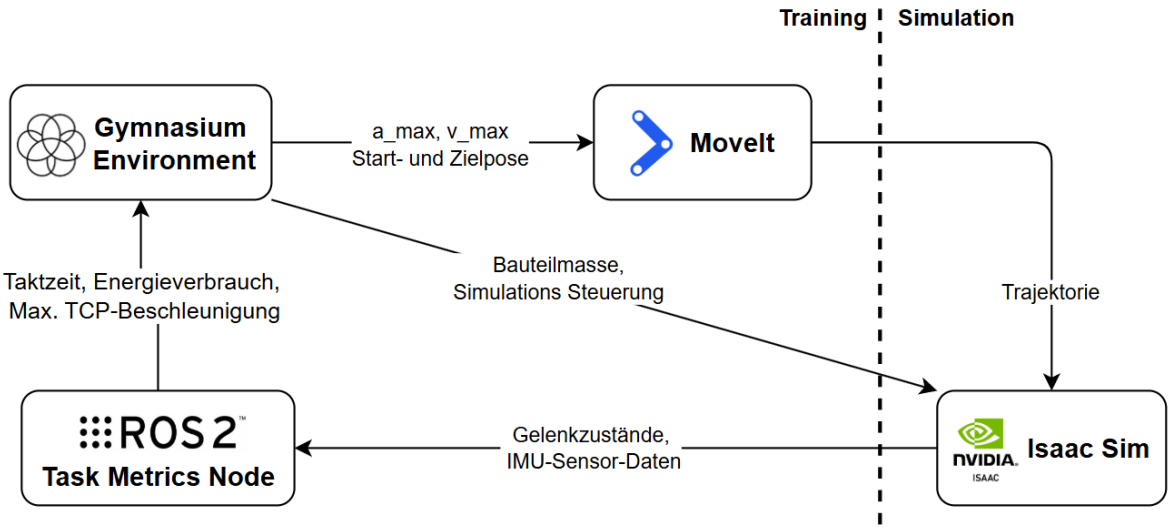


Abbildung 5.1: Systemarchitektur: Zusammenspiel der Komponenten während des Trainings.

geschlossen ist. Eine Integration beider Funktionen in dieselbe Klasse würde daher parallele Verarbeitung erfordern, die Modularität des Systems verringern und die Architektur verkomplizieren.

Die Gymnasium-Umgebung kapselt dabei die gesamte Trainingslogik, indem sie zentrale Prozesse wie das Zurücksetzen der Simulation, die Aufbereitung der Beobachtungen, die Ausführung der Aktionen sowie die Berechnung der Belohnungen übernimmt.

5.2.2 Datenfluss

Zu Beginn jeder Episode initialisiert die Gymnasium-Umgebung den Gesamtzustand des Systems. Dazu wird die Bauteilmasse in der Simulation gesetzt, der Roboter in die Startpose der ausgewählten Bewegung gefahren und der Planungskontext von *MoveIt* auf die aktuelle Startpose aktualisiert, sodass die nachfolgende Bewegungsplanung vom korrekten Ausgangszustand ausgeht. Anschließend erhält der Agent eine Beobachtung, die die Charakteristik der auszuführenden Bewegung sowie die aktuellen Bauteilparameter beinhaltet. Auf dieser Basis leitet der Agent die Dynamikparameter `max_acc` und `max_vel` ab. Diese werden zusammen mit der Zielbewegung an *MoveIt* übergeben, das die Trajektorie plant und über ROS2 an *Isaac Sim* übermittelt. Dadurch wird die in Anforderung A1.2 geforderte Parameterübergabe realisiert, da die vom Agenten gesetzten Dynamikparameter unmittelbar in die Bewegungsplanung integriert werden und somit die Ausführung der Bewegung in der Simulation direkt beeinflussen. Während der Ausführung erfasst der *Task Metrics Node* kontinuierlich Gelenkzustände und IMU-Daten. Nach Abschluss der Bewegung stellt er der Gymnasium-Umgebung Zykluszeit, Energieverbrauch und maximale Beschleunigungen am *Tool Center Point* (TCP) bereit, auf deren Basis dann der *Reward* berechnet wird.

5.2.3 Simulation der Bauteilmasse

Um variierende Bauteilmassen ohne explizite Greif- und Ablegesequenz zu simulieren wird, wie in Abbildung 5.2 dargestellt, am TCP eine virtuelle Kugel platziert. Diese ist über eine starre, nichtaktuierte Verbindung ohne eigene Freiheitsgrade, mit dem Roboterflansch verbunden, sodass sie

sich stets synchron mit dem Endeffektor bewegt und ihre Trägheitskräfte vollständig auf den Roboter übertragen werden.

Da das System ausschließlich die Dynamikparameter vorgegebener Bewegungen optimiert und die Bauteilhandhabung nicht physikalisch simuliert, sondern rechnerisch über das Reibmodell in Abschnitt 5.2.4 geprüft wird, ist kein vollständiger Greifvorgang erforderlich. Dies vermeidet die notwendigen Planungs- und Ausführungsschritte eines realen Greifvorgangs nach jedem Simulations-Reset und reduziert die Trainingsdauer erheblich.

Zu Beginn jeder Episode wird die Masse der Kugel neu gesetzt, sodass der Agent lernt, `max_vel` und `max_acc` an die jeweilige Nutzlast anzupassen.

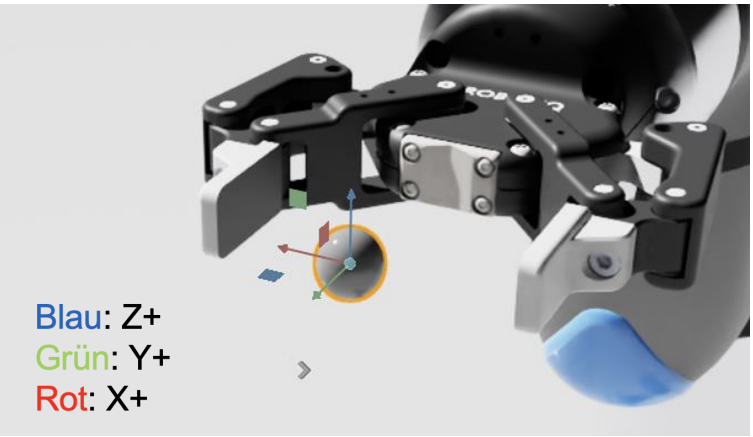


Abbildung 5.2: Virtuelle Masse am TCP: Zeigt die Position der virtuellen Masse in *Isaac Sim*.

5.2.4 Bauteilschlupf-Berechnung

Die Sicherstellung der korrekten Bauteilhandhabung basiert auf dem Coulomb'schen Reibmodell. Dieses besagt, dass ein Objekt solange haftend bleibt, wie die angreifende Tangentialkraft die maximal übertragbare Haftreibungskraft nicht überschreitet:

$$F_{\text{tangential}} \leq \mu \cdot F_{\text{normal}} \tag{5.1}$$

Wobei μ den Reibkoeffizienten zwischen Greifer und Bauteil und F_{normal} die senkrecht auf die Kontaktfläche wirkende Greifkraft bezeichnet. Da die Tangentialkraft nach dem zweiten Newton'schen Gesetz mit $F_{\text{tangential}} = m \cdot a_{\text{mag}}$ aus der Bauteilmasse m und dem Beschleunigungsbetrag am (TCP) folgt und der IMU-Sensor zusätzlich die Erdbeschleunigung erfasst, lässt sich Gleichung 5.1 direkt nach der maximal zulässigen Beschleunigung auflösen:

$$a_{\text{max,zul}} = \frac{\mu \cdot 2\, F_{\text{grip}}}{m} \tag{5.2}$$

Wobei F_{grip} die einseitige Greifkraft ist und der Faktor zwei die beidseitige Einspannung des Greifers berücksichtigt. Dieser Wert wird dem Agenten als Eingangsgröße übergeben. Der Agent lernt, die

Dynamikparameter so zu wählen, dass der tatsächlich auftretende Beschleunigungsbetrag am TCP diesen Grenzwert nicht überschreitet.

Die Beschleunigung wird über den IMU-Sensor auf der virtuellen Masse erfasst. Da der Sensor das gleiche Koordinatensystem wie die Kugel hat, gibt er die Beschleunigungen direkt in diesem Koordinatensystem aus. Als Beschleunigungsbetrag wird die vollständige räumliche Beschleunigung verwendet:

$$a_{\text{mag}} = \|\mathbf{a}_{\text{raw}}\| = \sqrt{a_x^2 + a_y^2 + a_z^2} \tag{5.3}$$

Die Verwendung aller drei Raumrichtungen hat einen entscheidenden Vorteil. Im Training wird stets dieselbe Greiferorientierung verwendet, weshalb nicht bekannt ist, welche Achsen bei anderen Orientierungen die dominanten Beschleunigungskomponenten darstellen. Durch die Verwendung des vollen Beschleunigungsbetrags wird implizit der ungünstigste Fall abgedeckt, sodass das trainierte Modell unabhängig von der späteren Greiferorientierung eingesetzt werden kann. Die Schlupfbedingung lautet damit:

$$a_{\text{mag}} \geq a_{\text{max,zul}} \tag{5.4}$$

Wird dieser Grenzwert überschritten, erhält der Agent einen negativen *Reward* , wodurch er lernt, `max_acc` und `max_vel` so zu wählen, dass die Bauteilhandhabung sichergestellt ist.

Diese konservative Betrachtung hat zur Folge, dass das trainierte Modell tendenziell vorsichtigere Parameter wählt als bei einer orientierungsspezifischen Auswertung möglich wäre, was eine Einschränkung der erreichbaren Optimalität bedeutet. Dies steht jedoch im Einklang mit der in Anforderung A3.1 definierten Priorität, wonach Optimalität nur unter der Bedingung angestrebt wird, dass die Sicherheitsanforderungen A2 erfüllt sind.

5.3 Modellierung als Markov Decision Process

Die Lernaufgabe wird als MDP modelliert, der durch das Tupel (S, A, R, γ) beschrieben wird, wobei S den Zustandsraum, A den Aktionsraum, R die *Reward*-Funktion und γ den *Discount-Factor* bezeichnet (Sutton & Barto, 2018). Ein MDP beschreibt die Interaktion eines Agenten mit einer Umgebung als sequentielle Entscheidungsaufgabe. Der Agent beobachtet einen Zustand, wählt eine Aktion, erhält einen *Reward* und geht in einen neuen Zustand über. Im vorliegenden System entspricht ein Zustand dem Beobachtungsvektor der aktuellen Bewegungsaufgabe, eine Aktion den gewählten Dynamikparametern und der *Reward* der gemessenen Ausführungsqualität. Ein MDP setzt voraus, dass die Entscheidung des Agenten ausschließlich auf dem aktuellen Zustand basiert und keine Kenntnis vergangener Zustände erfordert. Diese Eigenschaft ist im vorliegenden System erfüllt, da alle für die Parameterwahl relevanten Informationen vollständig im Beobachtungsvektor vorhanden sind. Da jede Episode aus genau einem Bewegungsschritt besteht und unmittelbar nach der Ausführung endet, hat γ keinen Einfluss auf das Lernverhalten und wird daher nicht weiter betrachtet.

5.3.1 Zustandsraum

Der Zustandsraum S beschreibt die Menge aller Informationen, die dem Agenten zur Verfügung stehen, um eine Entscheidung über die Dynamikparameter zu treffen. Der Beobachtungsvektor $\mathbf{s} \in \mathbb{R}^9$ setzt sich aus drei Gruppen von Merkmalen zusammen, wie in Tabelle 5.1 dargestellt.

Tabelle 5.1: Beobachtungsvektor des RL-Agenten

Index	Merkmal	Wertebereich	Bedeutung
o_1	`is_ptp`	$\{0, 1\}$	Bewegungstyp PTP
o_2	`is_lin`	$\{0, 1\}$	Bewegungstyp LIN
o_3	`dist_joints`	$[0, 8]$ rad	Summierte Gelenkwinkeländerung
o_4	`dist_cart`	$[0, 2]$ m	Kartesische Distanz Start zu Ziel
o_5	`delta_z`	$[-1, 1]$ m	Höhenänderung des TCP
o_6	`energy_weight`	$[0, 1]$	Gewichtungsfaktor von Energie und Taktzeit
o_7	`is_loaded`	$\{0, 1\}$	Beladungszustand
o_8	`current_mass`	$[0.1, 12]$ kg	Bauteilmasse
o_9	`accel_budget`	$[13, 22]$ m/s^2	Maximal zulässige Beschleunigung $a_{\text{max,zul}}$

Die ersten fünf Merkmale beschreiben die kinematische Charakteristik der auszuführenden Bewegung. Der Agent erhält dabei bewusst keine absoluten Gelenkwinkel oder kartesischen Posen, sondern ausschließlich bewegungscharakterisierende Merkmale wie summierte Gelenkwinkeländerung, kartesische Distanz und Höhenänderung des TCP. Dadurch erzeugen zwei strukturell ähnliche Bewegungen, die sich lediglich in ihrer absoluten Position im Arbeitsraum unterscheiden, ähnliche Beobachtungsvektoren und erhalten vom Agenten ähnliche Dynamikparameter. Diese Eigenschaft des Beobachtungsraums ist die Grundlage für die Generalisierung auf nicht im Training enthaltene Bewegungen und ermöglicht es, ein Modell zu trainieren, das auf einer repräsentativen Teilmenge der an einer Station vorkommenden Bewegungen lernt und die erlernten Zusammenhänge auf strukturell ähnliche Bewegungen derselben Station überträgt.

Der Gewichtungsfaktor `energy_weight` beschreibt die prozessseitige Zielvorgabe und ermöglicht dem Agenten, ohne Neu-Training zwischen Energie- und Zeitoptimierung zu wechseln. Die bauteilbezogenen Merkmale `is_loaded`, `current_mass` und `accel_budget` stellen sicher, dass der Agent die Sicherheitsanforderung A2.3 in seine Parameterwahl einbezieht.

5.3.2 Aktionsraum

Der Aktionsraum A ist kontinuierlich und zweidimensional:

$$\mathbf{a} = (\texttt{max_vel},\ \texttt{max_acc}) \in [0.01,\ 1.0]^2 \tag{5.5}$$

Beide Parameter sind relative Skalierungsfaktoren, deren Bedeutung vom Bewegungstyp abhängt. Bei PTP-Bewegungen skalieren sie die roboterspezifischen Maximalwerte für Gelenkgeschwindigkeit und Gelenkbeschleunigung. Bei LIN-Bewegungen beschreiben die Parameter hingegen, wie schnell die kartesische Bahn verfahren wird.

5.3.3 *Reward*-Funktion

Die *Reward*-Funktion verankert die Anforderungen aus Kapitel 4 direkt im Lernprozess. Sie ist in drei Prioritätsstufen aufgebaut, die der Reihenfolge Sicherheit vor Optimalität entsprechen.

Die höchste Priorität hat die Bewegungsausführung selbst. Kann *MoveIt* die Trajektorie nicht planen oder ausführen, erhält der Agent einen stark negativen *Reward*, der ihn dazu bringt, Parameter zu wählen die zu einer erfolgreichen Ausführung führen. Bei PTP-Bewegungen ist dies durch die interne Begrenzung auf die Hardwaregrenzen ausgeschlossen. Bei LIN-Bewegungen hingegen skalieren die Parameter die kartesische Bahngeschwindigkeit, woraus der Planer die erforderlichen Gelenkgeschwindigkeiten erst ableitet. Bei zu hohen Parameterwerten können diese die Hardwaregrenzen überschreiten und der Planer lehnt die Bewegung ab. Der Agent lernt dadurch, für LIN-Bewegungen konservativere Parameter zu wählen als für PTP-Bewegungen.

Die zweite Priorität gilt der Prozesssicherheit. Überschreitet der gemessene Beschleunigungsbetrag am TCP die maximal zulässige Beschleunigung $a_{\mathrm{max,zul}}$, verrutscht das Bauteil im Greifer. Dies gefährdet nicht nur die Handhabung des Bauteils selbst, sondern auch die Wiederholgenauigkeit des gesamten Prozesses, da ein verrutschtes Bauteil in einer nicht definierten Position gehalten wird und damit das nachfolgende Platzieren des Bauteils fehlschlagen kann. Der Agent wird daher proportional zur Überschreitung bestraft. Dies hat den Vorteil, dass der Agent erkennt, dass eine geringe Überschreitung besser ist als eine starke, und so schrittweise die Einhaltung der zulässigen Beschleunigung erlernt.

Sind beide Sicherheitsbedingungen erfüllt, erhält der Agent eine positive Basisbelohnung, die durch Verbesserungen in Zykluszeit oder Energieverbrauch gegenüber eines Referenzmodells erhöht wird. Das Referenzmodell beschreibt das Verhalten des Roboters bei festen Referenzparametern und dient als Vergleichsmaßstab für beide Metriken. Die genaue Definition des Referenzmodells sowie die Berechnung der Verbesserungsanteile werden in Kapitel 6.3 beschrieben. Die Gewichtung zwischen Zeit- und Energieverbesserung wird durch den Parameter `energy_weight` gesteuert, sodass der Agent je nach Zielvorgabe unterschiedliche Optimierungsstrategien erlernt. Für LIN-Bewegungen wird eine abgeschwächte Gewichtung verwendet, da bei kartesisch-linearen Bewegungen die Durchführungssicherheit Vorrang vor der Optimierung hat und zu aggressive Parameter zu einer Ablehnung durch den Bewegungsplaner führen können. Die resultierende *Reward*-Funktion lässt sich damit als dreistufige Fallunterscheidung zusammenfassen: negativer *Reward* bei nicht ausführbarer Trajektorie, negativer *Reward* bei Bauteilschlupf, positive Basisbelohnung zuzüglich gewichteter Verbesserungen gegenüber des Referenzmodells bei erfolgreicher und sicherer Ausführung. Die konkrete Parametrierung dieser Stufen wird in Kapitel 6.2.3 beschrieben.

5.4 Bewegungsbibliothek

Die Trainingsbibliothek umfasst 13 Bewegungen, acht PTP- und fünf LIN-Bewegungen, die typischerweise in industriellen *Pick-and-Place* Anwendungen vorkommen. Die Auswahl deckt die wesentlichen kinematischen Grundtypen ab: horizontale Transferbewegungen, vertikale Hebe- und

Senkbewegungen, diagonale Trajektorien sowie Bewegungen mit Orientierungsänderung des Endeffektors, jeweils in verschiedenen Distanzbereichen. Die Auswahl erfolgte heuristisch mit dem Ziel, eine möglichst breite Abdeckung des kinematischen Merkmalsraums aus Gelenkdistanz, kartesischer Distanz und Höhenänderung zu erreichen. Da das System stationsbezogen ausgelegt ist und die auftretenden Bewegungen innerhalb einer Station eine hohe kinematische Ähnlichkeit aufweisen, stellt diese Bibliothek eine repräsentative Grundlage für das Training dar. Eine systematische Analyse der Merkmalsraumabdeckung ist nicht Gegenstand dieser Arbeit.

Kapitel 6 Implementierung

6.1 Simulationsumgebung

Die Simulationsumgebung basiert auf *Isaac Sim* 5.1, das über die ROS2-*Bridge* und die *Simulation Control Extension* mit dem ROS2-Netzwerk verbunden ist. Der Roboter ist sowohl in *Isaac Sim* als USD-Szene hinterlegt als auch in *MoveIt* als URDF-Modell inklusive Kollisionsgeometrie eingebunden. Der simulierte Roboter ist ein UR30 mit sechs Freiheitsgraden, der als typischer Industrieroboter eine repräsentative Grundlage für die Evaluation bildet und über eine ROS2- und *MoveIt*-Integration verfügt.

MoveIt wird mit dem *Pilz Industrial Motion Planner* als primärer Planer betrieben (Pilz GmbH & Co. KG, 2024). Da gleiche Eingangsparameter stets identische Bewegungen erzeugen müssen um eine valide Evaluation zu gewährleisten, wird ein deterministischer Planer benötigt. Die Auswahl des Bewegungsplaners ist in Tabelle 6.1 begründet.

Tabelle 6.1: Vergleich der *MoveIt* Bewegungsplaner

Planer	Trajektorie	Deterministisch	Bewertung
OMPL [1]	Gelenkraum, sampling-basiert	Nein	Nicht reproduzierbar, ungeeignet für valide Evaluation
CHOMP [2]	Gelenkraum, gradientenbasiert	Nein	Optimierungsbasiert, konvergiert nicht immer, kein industrietypisches Verhalten
STOMP [3]	Gelenkraum, stochastisch	Nein	Stochastisch, ähnliche Einschränkungen wie OMPL
Pilz [4]	PTP: Gelenkraum, LIN: Kartesisch	Ja	Deterministisch, industrietypisch, direkte Unterstützung von PTP und LIN, gewählt für diese Arbeit

Der Pilz Industrial Motion Planner unterstützt die Bewegungstypen PTP für gelenkraum basierende Bewegungen sowie LIN für kartesisch-lineare Trajektorien. Die Planungsanfragen werden über eine *Action-Server* entgegengenommen, der den Bewegungstyp sowie die Skalierungsfaktoren `max_vel` und `max_acc` als Eingangsgrößen akzeptiert. Bei PTP-Bewegungen skalieren diese Faktoren die

[1] https://moveit.picknik.ai/main/doc/how_to_guides/using_ompl_constrained_planning/ompl_constrained_planning.html

[2] https://moveit.picknik.ai/main/doc/how_to_guides/chomp_planner/chomp_planner_tutorial.html

[3] https://moveit.picknik.ai/main/doc/how_to_guides/stomp_planner/stomp_planner.html

[4] https://moveit.picknik.ai/main/doc/how_to_guides/pilz_industrial_motion_planner/pilz_industrial_motion_planner.html

roboterspezifischen Maximalwerte für Gelenkgeschwindigkeit und Gelenkbeschleunigung. Bei LIN-Bewegungen skalieren sie die konfigurierten kartesischen Limits, die auf Basis des *Universal Robots ROS2 Driver Packages* gesetzt sind.

6.2 Gymnasium-Umgebung

Die Gymnasium-Umgebung ist als ROS2-Node implementiert, sodass die Kommunikation mit den anderen Systemkomponenten direkt über *Topics* und *Services* erfolgen kann, ohne eine zusätzliche Kommunikationsschicht zu benötigen. Sie bildet die zentrale Schnittstelle zwischen dem RL-Agenten und der Simulationsumgebung und kapselt die gesamte Trainingslogik. Die Umgebung besteht aus vier wesentlichen Komponenten: der Bewegungsbibliothek, die alle trainierbaren Bewegungen samt vorberechneter Merkmale enthält, dem *Reset*-Mechanismus, der zu Beginn jeder Episode den Roboter in einen definierten Ausgangszustand versetzt und die Aufgabenparameter randomisiert sowie der *Step*-Funktion, die einen vollständigen Trainingsschritt von der Parameterübergabe bis zur *Reward*-Berechnung umsetzt.

6.2.1 Bewegungsbibliothek

Die Bewegungsbibliothek ist als Python-Liste implementiert, in der jede Bewegung als Tupel aus Startgelenkwinkel, Zielgelenkwinkel, Bewegungstyp und Name hinterlegt ist. Die Posen werden bewusst im Gelenkraum definiert, da Gelenkraumziele eindeutig sind und keine Inversekinematik-Berechnung zur Laufzeit erfordern. Dies macht die Bewegungsausführung deterministischer und vermeidet Mehrdeutigkeiten die bei kartesischen Zielen durch mehrere gültige Gelenkwinkelkonfigurationen entstehen können.

Bei der Umgebungsinitialisierung werden für jeden Bewegung die kartesischen Posen von Start- und Zielkonfiguration berechnet. Daraus werden die Beobachtungsmerkmale kartesische Distanz zwischen Start- und Zielpose, summierte Gelenkwinkeländerung sowie Höhenänderung des TCP vorberechnet und als feste Einträge zur entsprechenden Bewegung hinterlegt, sodass während des Trainings keine kinematischen Berechnungen mehr erforderlich sind.

6.2.2 *Reset*-Mechanismus

Zu Beginn jeder Episode werden zunächst die Zustandsparameter zufällig bestimmt. Der Beladungszustand wird gleichverteilt ausgewählt. Bei beladenem Zustand werden Bauteilmasse aus dem Intervall $[0.1,\ 12]\,\mathrm{kg}$ und die maximal zulässige Beschleunigung $a_{\mathrm{max,zul}}$ aus dem Intervall $[13,\ 22]\,\mathrm{m/s^2}$ entnommen. Die obere Grenze der Bauteilmasse ergibt sich aus der maximalen Greifkraft des eingesetzten Greifers. Als ungünstigster Reibkoeffizient wird $\mu = 0.3$ angenommen, was einem konservativen Schätzwert unterhalb typischer Metall-Gummi Haftreibungswerte entspricht und damit einen realistischen unteren Grenzwert bildet. Setzt man diesen Wert zusammen mit der maximalen Greifkraft $F_{\mathrm{grip}} = 235\,\mathrm{N}$ in Gleichung 5.2 ein und betrachtet den statischen Fall mit $a_{\mathrm{mag}} = g$, ergibt sich eine maximale Bauteilmasse von:

$$m_{\max} = \frac{\mu \cdot 2 \cdot F_{\mathrm{grip}}}{g} = \frac{0.3 \cdot 2 \cdot 235}{9.81} \approx 14.4\,\mathrm{kg} \tag{6.1}$$

Bei dieser Masse wäre das Bauteil bereits im statischen Fall an der Haftgrenze, sodass jede Roboterbewegung durch die zusätzlich auftretenden Beschleunigungen zwangsläufig zu einem Verrutschen des Bauteils führen würde. Der Trainingsbereich wird daher konservativ auf 12 kg begrenzt, da höhere Bauteilmassen im späteren Einsatz dazu führen würden, dass $a_{\text{max,zul}}$ so klein wird, dass der Agent keine sinnvollen Dynamikparameter mehr ableiten kann und damit keine praxisrelevanten Anwendungsfälle darstellen. Die Grenzen des Intervalls für $a_{\text{max,zul}}$ wurden durch Testmessungen bestimmt, indem Bewegungen mit minimaler und maximaler Beschleunigung ausgeführt wurden. Die dabei gemessenen TCP-Beschleunigungen definieren den physikalisch erreichbaren Wertebereich des Systems und wurden als untere beziehungsweise obere Intervallgrenze übernommen.

Nachdem der Wert der Masse bestimmt wurde, wird dieser als die virtuelle Masse am TCP in *Isaac Sim* gesetzt. Danach wird eine zufällige Bewegung ausgewählt, und ihre Parameter dem Zustandsraum übergeben. Aus diesen und den Bauteilparametern ergibt sich dann der vollständige Beobachtungsvektor der Episode.

Der eigentliche *Reset*-Ablauf pausiert zunächst die Simulation und übermittelt die Startpose der aktiven Bewegung direkt als Soll-Trajektorie an den Gelenkregler. Dieser Ansatz wird gewählt weil ein direktes Setzen der Gelenkwinkel über den Regler deutlich schneller ist als eine vollständige Bewegungsplanung über *MoveIt* und zudem sicherstellt dass *Isaac Sim* den Roboter zuverlässig in die gewünschte Startpose bringt. Anschließend wird der Planungskontext von *MoveIt* über den `apply_planning_scene` Service auf die aktuelle Startpose aktualisiert. Damit befinden sich Simulation, Regler und Bewegungsplaner einheitlich im definierten Ausgangszustand und die nachfolgende Bewegungsplanung kann vom korrekten Startzustand ausgehen. Nach erfolgreichem Abschluss wird die Simulation wieder gestartet.

6.2.3 *Step*-Funktion

Die *Step*-Funktion setzt den Ablauf eines Trainingsschritts um. Zunächst wird die Energiemessung im *Task Metrics Node* zurückgesetzt. Anschließend wird die aktive Bewegung mit den vom Agenten gewählten Parametern `max_vel` und `max_acc` an *MoveIt* übergeben und ausgeführt. Nach Abschluss der Bewegung werden die berechneten Metriken abgefragt. Auf Basis dieser Werte wird dann der *Reward* berechnet.

Zunächst wird geprüft ob die Bewegung erfolgreich ausgeführt wurde. Ist dies nicht der Fall, wird ein fester negativer *Reward* von -15.0 vergeben. Wurde die Bewegung erfolgreich ausgeführt, wird anschließend geprüft ob der gemessene Beschleunigungsbetrag a_{mag} die maximal zulässige Beschleunigung $a_{\text{max,zul}}$ überschreitet. Bei einem Slip-Ereignis wird ein negativer *Reward* zwischen -10.0 und -20.0 vergeben, der proportional zur Überschreitung skaliert:

$$r_{\text{slip}} = -10.0 - 2.0 \cdot \min\left(a_{\text{mag}} - a_{\text{max,zul}},\ 5.0\right) \tag{6.2}$$

Sind beide Sicherheitsbedingungen erfüllt, wird eine positive Basisbelohnung von 5.0 durch gewichtete Verbesserungen gegenüber dem Referenzmodell erhöht. Für PTP-Bewegungen fließen dabei sowohl Zeitverbesserung als auch Energieverbesserung mit den durch `energy_weight` bestimmten Gewichten ein. Für LIN-Bewegungen wird eine abgeschwächte Gewichtung verwendet, da kartesisch-lineare Bewegungen anfälliger für Ausführungsfehler wie Bauteilschlupf, Verletzung von Gelenkgrenzen oder kinematische Singularitäten sind, weshalb die Durchführungssicherheit hier stärker gegenüber der Optimierung gewichtet wird.

$$r = \begin{cases} -15.0 & \text{Bewegung nicht ausführbar} \\ r_{\text{slip}} & a_{\text{mag}} > a_{\text{max,zul}} \\ 5.0 + w_e \cdot \Delta e + w_t \cdot \Delta t & \text{sonst, PTP} \\ 5.0 + 0.2 \cdot \Delta e + 0.2 \cdot \Delta t & \text{sonst, LIN} \end{cases} \tag{6.3}$$

wobei Δe die relative Energieverbesserung, Δt die relative Zeitverbesserung gegenüber dem Referenzmodell, $w_e = \texttt{energy_weight} \cdot 10$ und $w_t = (1 - \texttt{energy_weight}) \cdot 10$ die zugehörigen Gewichte bezeichnen.

6.3 Referenzmodell und Verbesserungsberechnung

Das Referenzmodell beschreibt das Verhalten des Roboters bei festen Referenzparametern von $\texttt{max_vel} = 0.5$ und $\texttt{max_acc} = 0.5$. Die Wahl der Referenzparameter entspricht der numerischen Mitte des Aktionsraums und stellt damit einen neutralen Ausgangspunkt dar. Ein Referenzmodell ist notwendig, da absolute Messwerte für Zykluszeit und Energieverbrauch zwischen verschiedenen Bewegungen nicht direkt vergleichbar sind. Eine lange Diagonalbewegung benötigt zwangsläufig mehr Zeit und Energie als eine kurze Vertikalbewegung, unabhängig von der Parameterwahl. Durch die Normierung auf die Werte des Referenzmodells lassen sich Verbesserungen verschiedener Bewegungen auf einer einheitlichen relativen Skala bewerten und auch Energie- und Zeitverbesserung werden dadurch direkt miteinander vergleichbar. Für jede Bewegung in der Bibliothek wurden Referenzwerte für Zykluszeit und Energieverbrauch durch wiederholte Messungen in der Simulation ermittelt. Da sich Zykluszeit und Energieverbrauch mit steigender Bauteilmasse verändern, wurden die Referenzwerte jeweils für den unbeladenen und den maximal beladenen Zustand getrennt gemessen. Zur Laufzeit wird der zutreffende Referenzwert durch lineare Interpolation anhand der aktuellen Bauteilmasse bestimmt:

$$t_{\text{ref}} = t_{\text{min}} + \frac{m - m_{\text{min}}}{m_{\text{max}} - m_{\text{min}}} \cdot (t_{\text{max}} - t_{\text{min}}) \tag{6.4}$$

$$e_{\text{ref}} = e_{\text{min}} + \frac{m - m_{\text{min}}}{m_{\text{max}} - m_{\text{min}}} \cdot (e_{\text{max}} - e_{\text{min}}) \tag{6.5}$$

Wobei t_{min} und t_{max} die gemessenen Zykluszeiten sowie e_{min} und e_{max} die gemessenen Energieverbrauchswerte bei unbeladenem und maximal beladenem Zustand bezeichnen. m beschreibt die aktuell in der Simulation gesetzte Bauteilmasse, während m_{min} und m_{max} die minimale beziehungsweise maximale im Training berücksichtigte Bauteilmasse darstellen. Die Verwendung einer linearen Interpolation ist dadurch begründet, dass Messungen bei minimaler, mittlerer und maximaler Bauteilmasse eine näherungsweise lineare Abhängigkeit des Energieverbrauchs von der Bauteilmasse gezeigt haben. Die Zykluszeit zeigte dabei eine deutlich geringere Abhängigkeit von der Bauteilmasse, da die im Training verwendeten Massen weit unterhalb der Traglastgrenze des Roboters liegen und die gewählten Geschwindigkeits- und Beschleunigungswerte damit unabhängig von der Last vollständig ausgefahren werden können. Die relativen Verbesserungen gegenüber des Referenzmodells werden dann für beide Metriken getrennt berechnet:

$$\Delta t = \frac{t_{\text{ref}} - t_{\text{agent}}}{t_{\text{ref}}}, \qquad \Delta e = \frac{e_{\text{ref}} - e_{\text{agent}}}{e_{\text{ref}}} \tag{6.6}$$

Positive Werte bedeuten dabei eine Verbesserung gegenüber des Referenzmodells, negative Werte eine Verschlechterung. Diese Verbesserungen fließen gewichtet durch `energy_weight` in den *Reward* ein, wie in Gleichung 6.3 beschrieben.

6.4 Trainingskonfiguration

Das Training wird mit der *Stable Baselines3* Implementierung des SAC-Algorithmus durchgeführt. Die gewählten Hyperparameter basieren auf den Standardwerten der Bibliothek und wurden dort angepasst wo die Problemstellung gezielte Einstellungen erfordert. Eine Übersicht aller verwendeten Parameter ist in Tabelle 6.2 dargestellt.

Tabelle 6.2: Hyperparameter der SAC-Trainingskonfiguration

Parameter	**Wert**	**Begründung**
Learning Rate	3×10^{-4}	Standardwert SAC
Replay Buffer Size	50 000	Ausreichend für *Off-Policy*-Lernen bei kurzen Episoden
Batch Size	128	Standardwert SAC
Learning Starts	1 000	Buffer füllen vor erstem Update
Discount-Factor γ	0.99	Standardwert SAC, da γ bei Ein-Schritt-Episoden keinen praktischen Einfluss hat
Soft-Update Rate τ	0.005	Standardwert SAC
Entropy Coefficient	automatisch	Automatische Anpassung durch SAC, kein manuelles Tuning nötig
Train Frequency	1	Nach jedem Schritt ein Update, passend zu Ein-Schritt-Episoden
Gradient Steps	1	Standardwert SAC
Training Steps	80 000	Empirisch bestimmt

Die Beobachtungen und Belohnungen werden durch *VecNormalize* normalisiert um das Training zu stabilisieren. Da die Eingangsmerkmale des Beobachtungsvektors sehr unterschiedliche Wertebereiche aufweisen, würden unnormalisierte Eingaben dazu führen dass einzelne Dimensionen das Lernverhalten dominieren. Die *Reward*-Normalisierung stellt sicher dass auch kleine Verbesserungen gegenüber des Referenzmodell ein ausreichend starkes Lernsignal erzeugen und nicht von episodischen Schwankungen überdeckt werden.

6.5 Einbindung in ein modulares Ausführungssystem

Das trainierte Modell bestimmt Dynamikparameter für einzelne Roboterbewegungen. Um es in realen Anwendungen einzusetzen, muss es an eine Ausführungsinfrastruktur angebunden werden, die diese Bewegungen tatsächlich ausführt. Als allgemeine Ausführungsstruktur dient das in Heuss et al. (2022) beschriebene modulare Ausführungssystem, das Roboteraufgaben in wiederverwendbare Bausteine unterteilt und deren Ausführung koordiniert. Durch die Einbindung des Modells in dieses System kann es für alle Anwendungen genutzt werden, die sich durch eine Sequenz dieser Bausteine beschreiben lassen, ohne dass eine anwendungsspezifische Anpassung des Modells erforderlich ist.

6.5.1 *Primitive* und *Composite Skills*

Das System unterscheidet zwischen zwei Arten von *Skills*. *Primitive Skills* sind grundlegende Roboteroperationen die nicht weiter unterteilt werden können. Für diese Arbeit relevante *Primitive Skills* sind `move_to_pose`, das den Roboter über eine PTP-Bewegung zu einer kartesischen Zielpose fährt, `move_linear`, das eine kartesisch-lineare Bewegung in einer definierten Richtung über eine gegebene Distanz ausführt, sowie `grip`, das den Greifer über einen `command`-Parameter öffnet oder schließt. Tabelle 6.3 zeigt die Parameter der einzelnen *Primitive Skills* im Überblick.

Tabelle 6.3: *Primitive Skills*, ihre Parameter und Funktionen

Skill	**Parameter**	**Bedeutung**	**Funktion**
`move_to_pose`	`x, y, z`	Zielposition in m	Fährt den Roboter über eine PTP-Bewegung zur angegebenen kartesischen Zielpose
	`q_1, q_2, q_3, q_4`	Zielorientierung als Quaternion	
`move_linear`	`direction_x, direction_y, direction_z`	Bewegungsrichtung als Einheitsvektor	Führt eine kartesisch-lineare Bewegung in der angegebenen Richtung über die gegebene Distanz aus
	`distance`	Distanz in mm	
`grip`	`command`	`open` oder `close`	Öffnet oder schließt den Greifer

Composite Skills setzen sich aus einer geordneten Sequenz von *Primitive Skills* zusammen und beschreiben damit eine vollständige Aufgabe. Ein typischer *Pick-and-Place* Vorgang besteht beispielsweise aus einer Anfahrbewegung zur Aufnahmeposition, dem Schließen des Greifers, einer Transferbewegung zur Ablageposition und dem anschließenden Öffnen des Greifers. Die Sequenz wird als strukturiertes *Dictionary* übergeben, das jeden *Primitive Skill* als Kind-Element enthält. Ein beispielhafter *Pick-and-Place* Vorgang könnte wie folgt aussehen:

```
{
  "type": "sequence",
  "children": [
    {"type": "action", "name": "move_to_pose",
     "parameters": {"x": x_pre, "y": y_pre, "z": z_pre,
                    "q_1": q1, "q_2": q2, "q_3": q3, "q_4": q4}},
```

```
    {"type": "action", "name": "move_linear",
     "parameters": {"direction_x": 0.0, "direction_y": 0.0,
                    "direction_z": -1.0, "distance": d_pick}},
    {"type": "action", "name": "grip",
     "parameters": {"command": "close"}},
    {"type": "action", "name": "move_linear",
     "parameters": {"direction_x": 0.0, "direction_y": 0.0,
                    "direction_z": 1.0, "distance": d_pick}},
    {"type": "action", "name": "move_to_pose",
     "parameters": {"x": x_place, "y": y_place, "z": z_place,
                    "q_1": q1, "q_2": q2, "q_3": q3, "q_4": q4}},
    {"type": "action", "name": "grip",
     "parameters": {"command": "open"}}
  ]
}
```

6.5.2 Einbindung des Modells

Um eine Vorhersage treffen zu können, benötigt das Modell alle neun Komponenten des in Tabelle 5.1 beschriebenen Beobachtungsvektors. Diese werden teils direkt aus dem *Skill-Dictionary* abgeleitet, teils müssen sie zusätzlich übergeben werden.

Aus dem *Skill-Dictionary* werden die bewegungscharakterisierenden Merkmale o_1 bis o_5 berechnet. Der Bewegungstyp (o_1, o_2) ergibt sich direkt aus dem *Skill*-Typ: `move_to_pose` entspricht einer PTP-Bewegung, `move_linear` einer LIN-Bewegung. Bei `move_to_pose` werden kartesische Distanz (o_4), summierte Gelenkwinkeländerung (o_3) und Höhenänderung des TCP (o_5) aus der aktuellen Pose und der im *Dictionary* angegebenen Zielpose berechnet. Bei `move_linear` wird zunächst aus Bewegungsrichtung und Distanz die kartesische Zielpose berechnet, woraus anschließend dieselben Merkmale abgeleitet werden.

Die verbleibenden Merkmale o_6 bis o_9 können nicht aus dem *Skill-Dictionary* abgeleitet werden und müssen daher zusätzlich übergeben werden. Der Gewichtungsfaktor `energy_weight` (o_6) wird als Prozessvorgabe übergeben. Für jedes zu greifende Bauteil werden Masse (o_8), Reibkoeffizient und maximale Greifkraft übergeben, woraus $a_{\text{max,zul}}$ (o_9) gemäß Gleichung 5.2 berechnet wird. Bei Aufgaben mit mehreren verschiedenen Bauteilen werden diese Parameter als geordnete Liste übergeben, wobei der erste Eintrag dem ersten gegriffenen Bauteil entspricht, der zweite dem zweiten und so weiter.

Der Beladungszustand (o_7) wird anhand des Greifer-Status im *Dictionary* bestimmt. Die Annahme dabei ist, dass jeder `grip`-Skill mit `command: close` das nächste Bauteil aus der Liste greift. Ab diesem Zeitpunkt gilt `is_loaded` $= 1$ mit den entsprechenden Bauteilparametern für alle nachfolgenden Bewegungen bis zum nächsten `grip`-Skill mit `command: open`. Auf dieser Basis sagt das Modell für jeden `move_to_pose` und `move_linear` Schritt `max_vel` und `max_acc` vorher, mit denen der *Composite Skills* anschließend ausgeführt werden kann.

Kapitel 7 Evaluation

7.1 Versuchsaufbau

Ziel der Evaluation ist es zu überprüfen, ob das trainierte Modell die in Kapitel 4 definierten Anforderungen erfüllt. Dazu wird das Modell sowohl auf bekannten Bewegungen aus der Trainingsbibliothek als auch auf unbekannten Bewegungen evaluiert, die strukturell ähnlich aber nicht identisch zu den Trainingsbewegungen sind. Letztere simulieren den Einsatz an einer Roboterstation, deren Bewegungen strukturell den Trainingsbewegungen ähneln, sich jedoch in den konkreten Start- und Zielposen unterscheiden.

7.1.1 Generierung unbekannter Bewegungen

Die unbekannten Testbewegungen werden durch Variation der bekannten Trainingsbewegungen erzeugt. Für jede Bewegung der Trainingsbibliothek wurde ein Datensatz von jeweils 20 ausführbaren Varianten mit zufällig generierten Start- und Zielposen erzeugt. Aus dieser Sammlung wurden für jeden der drei untersuchten Variationsradien ($r \in \{5, 10, 15\}\,\text{cm}$) jeweils 200 unbekannte Bewegungen stochastisch ausgewählt und evaluiert. Durch diese hohe Anzahl an lokalen Variationen pro Grundbewegung wird der kinematische Raum um die ursprünglichen Trajektorien engmaschig abgedeckt. Ein Erfolg bei diesen Testfällen belegt somit, dass das Modell die gelernten Zusammenhänge zuverlässig auf den gesamten Bereich innerhalb des jeweiligen Radius übertragen kann. Dies dient als Nachweis, dass der Agent eine robuste generalisierende Strategie zur Parameterwahl entwickelt hat.

Die neuen Start- und Zielposen werden durch eine zufällige Verschiebung der ursprünglichen Positionen erzeugt. Dabei wird für jede Pose zunächst eine zufällige Richtung im Raum bestimmt. Danach wird die Weite der Verschiebung, also der Radius, festgelegt. Aus dieser Kombination von Richtung und Radius ergibt sich ein Verschiebungsvektor, mit dem die ursprüngliche Pose versetzt wird.

Damit die Testposen dabei gleichmäßig im gesamten Raum verteilt sind und sich nicht zu stark in der Mitte ansammeln, wird der Radius kubisch berechnet (siehe Gleichung 7.1). Dies stellt sicher, dass das gesamte Volumen der Kugelschale für die Evaluation realistisch abgedeckt wird.

$$r = \left(u \cdot (r_{\text{max}}^3 - r_{\text{min}}^3) + r_{\text{min}}^3\right)^{1/3} \tag{7.1}$$

Eine gleichförmige Verteilung des Radius würde dazu führen dass Posen nahe dem Innenradius überrepräsentiert sind, da das Volumen einer Kugelschale kubisch mit dem Radius wächst. Durch die kubische Transformation wird dies korrigiert.

Die Wahl der Variationsradien erfolgt mit dem Ziel, die Generalisierungsfähigkeit des Agenten im Sinne der Anforderung A4 systematisch zu untersuchen. Dabei wird bewusst auf lokale Variationen der Start- und Zielposen fokussiert, da der Agent nicht auf beliebige Bewegungen im gesamten Arbeitsraum generalisieren soll, sondern auf strukturell ähnliche Bewegungen innerhalb einer Roboterstation.

Die gewählten Radien von 5 cm, 10 cm und 15 cm definieren dabei eine gestufte Abweichung von den Trainingsbewegungen. Während ein Radius von 5 cm nur geringe Variationen darstellt und somit primär die Interpolationsfähigkeit des Modells überprüft, führen größere Radien zu zunehmend stärkeren Abweichungen im Beobachtungsraum. Insbesondere beeinflussen die Verschiebungen die Merkmale der kartesischen Distanz, der Gelenkdistanz sowie der Höhenänderung, auf deren Basis der Agent seine Parameterwahl trifft.

Durch die Staffelung der Radien kann somit analysiert werden, wie robust das Modell auf zunehmende Abweichungen von den Trainingsdaten reagiert und bis zu welchem Grad eine zuverlässige Generalisierung möglich ist.

Die Größenordnung der gewählten Radien orientiert sich zudem an realistischen Variationen innerhalb industrieller Roboteranwendungen, die beispielsweise durch Positioniertoleranzen, unterschiedliche Bauteillage oder Anpassungen im Layout entstehen und typischerweise im Zentimeterbereich liegen. Größere Variationen würden hingegen zu Bewegungen führen, die sich in ihrer kinematischen Charakteristik deutlich von den Trainingsdaten unterscheiden und damit nicht mehr dem Ziel der stationsbezogenen Generalisierung entsprechen.

7.1.2 Evaluierte Metriken

Das Modell wird anhand von drei zentralen Metriken bewertet, die direkt auf die in Kapitel 4 definierten Anforderungen referenzieren. Diese Metriken erlauben eine quantitative Beurteilung, ob der Agent sowohl die harten Sicherheitsvorgaben als auch die weichen Optimierungsziele erfüllt.

Erfolgsrate (Sicherheit und Generalisierung) Die Erfolgsrate ist die primäre Kennzahl zur Verifikation der Anforderungen A2 (Sicherheit) und A4 (Adaptivität). Eine Bewegung gilt nur dann als erfolgreich, wenn sie zwei Kriterien gleichzeitig erfüllt:

- **Planungserfolg:** Die Trajektorie muss vom Pfadplaner erfolgreich berechnet und ohne Hardware-Grenzverletzungen oder Kollisionen ausgeführt werden (A2.1, A2.2).
- **Schlupffreiheit:** Die während der Bewegung gemessene Spitzenbeschleunigung a_{max} darf das zulässige Budget $a_{max,zul}$ nicht überschreiten, um ein Verrutschen des Bauteils zu verhindern (A2.3).

Durch die Auswertung dieser Erfolgsrate über die verschiedenen Variationsradien ($r = 5/10/15\,\text{cm}$) wird zudem die Anforderung der Generalisierung (A4.1) überprüft. Ein stabiler Erfolg bei steigenden Radien belegt, dass der Agent die Sicherheitsstrategie auf unbekannte Posen übertragen kann.

Relative Zeit- und Energieverbesserung (Optimalität) Zur Überprüfung von Anforderung A3.1 (Optimalität) werden die relative Zeitverbesserung (Δt) und die relative Energieverbesserung (Δe) herangezogen. Diese beschreiben, wie stark der Agent die jeweilige Metrik gegenüber einem Referenzmodell verbessert (siehe Gleichung 6.6).

7.2 Sicherheitsevaluation

Die Sicherheitsevaluation überprüft, ob das Modell Anforderung A2.3 erfüllt, also ob es die maximal zulässige Beschleunigung $a_{\text{max,zul}}$ zuverlässig einhält und damit Bauteilschlupf vermeidet. Um die Robustheit des Modells unter möglichst kritischen Bedingungen zu beurteilen, wird der untere Grenzwert $a_{\text{max,zul}} = 13\,\text{m/s}^2$ als Testgrenze gewählt. Als Vergleichsmaßstab dient das Referenzmodell mit festen Parametern `max_vel` $= 0.5$ und `max_acc` $= 0.5$. Abbildung 7.1 zeigt die Erfolgsrate aufgeteilt nach Erfolg, Ausführungsfehlern und Bauteilschlupf für bekannte Bewegungen sowie generierte Bewegungen mit Radien $r \in \{5, 10, 15\}\,\text{cm}$.

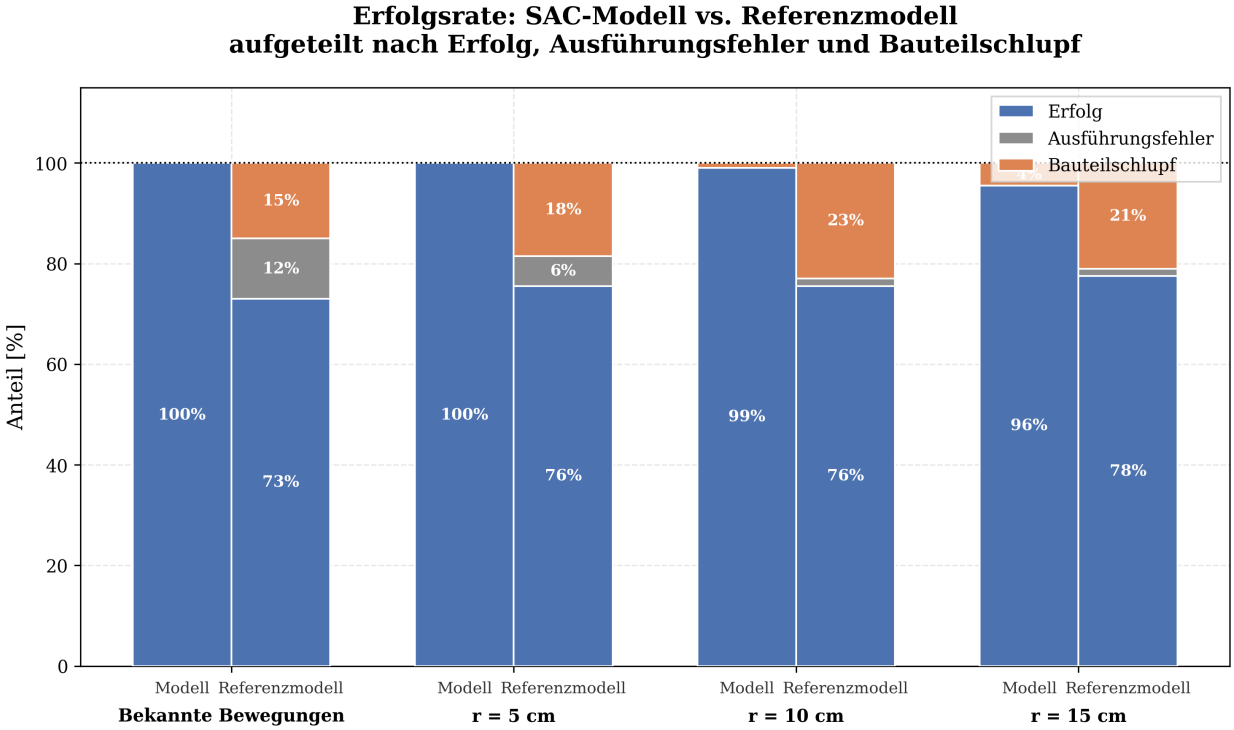


Abbildung 7.1: Erfolgsrate SAC-Modell vs. Referenzmodell: Vergleicht die Erfolgsrate bei $a_{\text{max,zul}} = 13.0\,\text{m/s}^2$ des Referenzmodells und des SAC-Modells für verschiedene Variationsradien.

Auf bekannten Bewegungen erreicht das Modell eine Erfolgsrate von 100 % ohne jegliche Slip-Ereignisse. Auf generierten Bewegungen bleibt die Erfolgsrate mit 100 % bei $r = 5\,\text{cm}$, 99 % bei $r = 10\,\text{cm}$ und 96 % bei $r = 15\,\text{cm}$ durchgehend hoch. Der Agent hat bei diesem Grenzwert ausreichend Trainingserfahrung gesammelt und wählt konsequent konservative Parameter. Bemerkenswert ist zudem, dass LIN-Bewegungen in allen Testszenarien fehlerfrei ausgeführt wurden. Da der Bewegungsplaner bei zu hohen kartesischen Geschwindigkeiten die Trajektorie ablehnt, hat der Agent für diesen Bewegungstyp eine besonders konservative Strategie erlernt. Abbildung 7.2 verdeutlicht dies. LIN-Bewegungen werden durchgehend mit $a_{\text{scale}} < 0.45$ ausgeführt, während der Agent bei PTP-Bewegungen deutlich höhere Beschleunigungen wählt. Das Referenzmodell erreicht hingegen nur 73 % bis 78 % Erfolgsrate mit einem Slip-Anteil von 21 % bis 23 % sowie Ausführungsfehlern von bis zu 18 %.

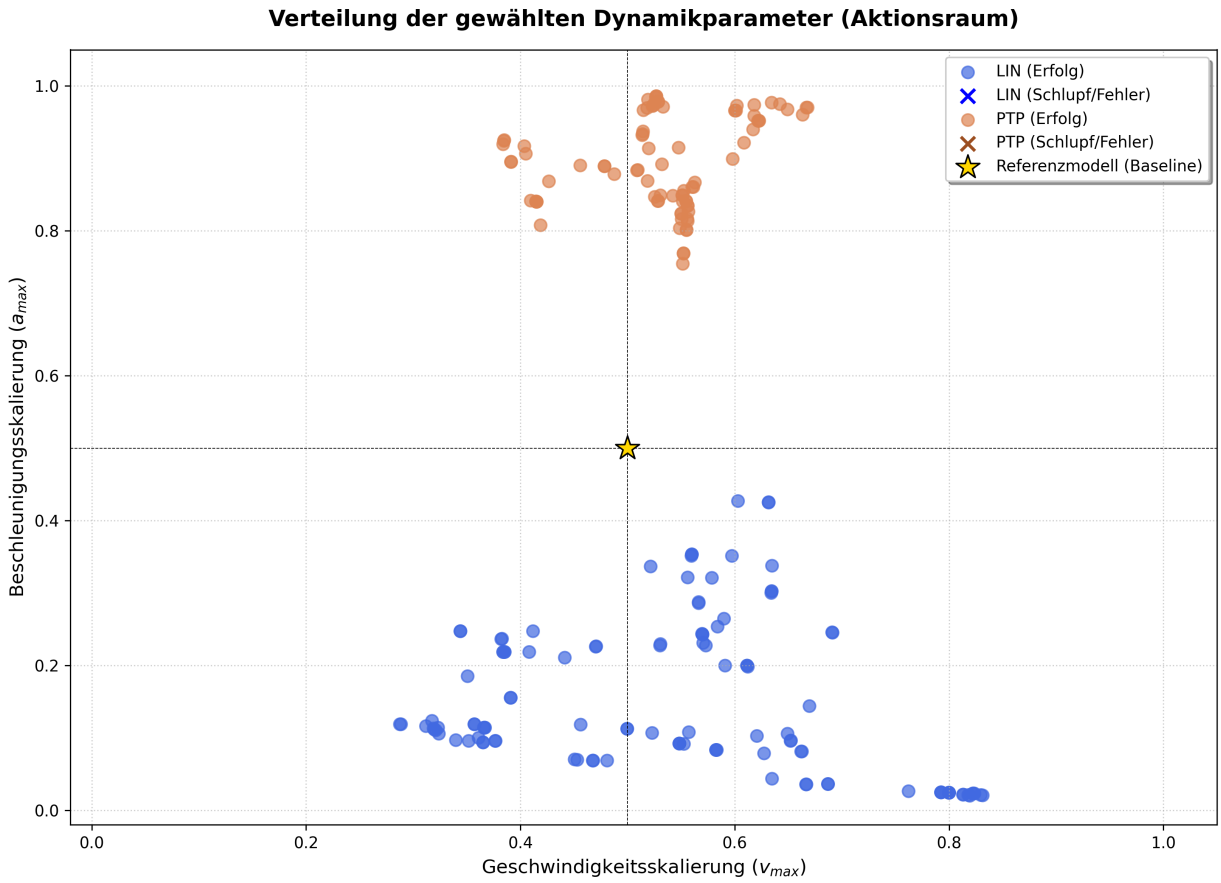


Abbildung 7.2: Verteilung der gewählten Dynamikparameter im Aktionsraum bei $a_{\text{max,zul}} = 13.0\,\text{m/s}^2$. LIN-Bewegungen (blau) werden konsequent mit niedrigen Beschleunigungsskalierungen ausgeführt, PTP-Bewegungen (orange) aggressiver. Stern: Referenzmodell.

Bewertung

Das Modell erfüllt Anforderung A2.3 zuverlässig, mit Erfolgsraten zwischen 96 % und 100 % über alle getesteten Bewegungen. Die niedrigste gemessene Erfolgsrate von 96 % tritt dabei ausschließlich beim maximalen Variationsradius von $r = 15\,\text{cm}$ auf. Für den Bereich von $r = 10\,\text{cm}$, der typische Positionsvariationen innerhalb einer Roboterstation abdeckt, werden bereits 99 % erreicht, was die Anforderung A4.1 an die stationsbezogene Generalisierung klar erfüllt. Auch die 96 % bei $r = 15\,\text{cm}$ sind dabei positiv zu bewerten. Sie bedeuten, dass bei grundlegend neuen Bewegungen im Extremfall jede zwanzigste Bewegung eine manuelle Nachparametrierung erfordert. Dies reduziert den Parametrieraufwand gegenüber einer vollständig manuellen Vorgehensweise erheblich und erfüllt damit das übergeordnete Ziel der Arbeit. Sollte auch dieser Restaufwand nicht akzeptabel sein, bieten sich zwei Maßnahmen an: Zum einen ein Feintuning des Modells auf die neuen Bewegungen, zum anderen ein konservativer Sicherheitsfaktor bei der Vorberechnung von $a_{\text{max,zul}}$ gemäß Gleichung 5.2, der den effektiven Spielraum des Agenten erhöht.

7.3 Optimalitätsevaluation

Die Optimalitätsevaluation überprüft, ob das Modell Anforderung A3.1 erfüllt, also ob der Agent die Dynamikparameter in Abhängigkeit des Gewichtungsfaktors `energy_weight` zielgerichtet optimiert. Dazu werden die relative Taktzeit-Verbesserung Δt und die relative Energie-Verbesserung Δe gegenüber dem Referenzmodell gemäß Gleichung 6.6 ausgewertet. Positive Werte bedeuten dabei eine Verbesserung gegenüber dem Referenzmodell. Die Evaluation erfolgt zunächst im unbeladenen Zustand, um das grundlegende Optimierungsverhalten des Agenten isoliert von Sicherheitsanforderungen zu beurteilen. Anschließend wird das Verhalten im beladenen Zustand analysiert, um die Auswirkungen des Beschleunigungslimits auf den Optimierungsprozess zu untersuchen. PTP- und LIN-Bewegungen werden getrennt betrachtet, da der Agent für beide Bewegungstypen aufgrund der unterschiedlichen *Reward*-Gewichtung grundlegend verschiedene Strategien erlernt hat.

7.3.1 PTP-Bewegungen bei unbeladenem Zustand

Abbildung 7.3 zeigt die Ergebnisse für PTP-Bewegungen bei den Gewichtungsfaktoren `energy_weight` $=$ 0.1 (Taktzeit-Fokus) und `energy_weight` $= 0.9$ (Energie-Fokus) sowie den Variationsradien $r = 5\,\text{cm}$ und $r = 15\,\text{cm}$.

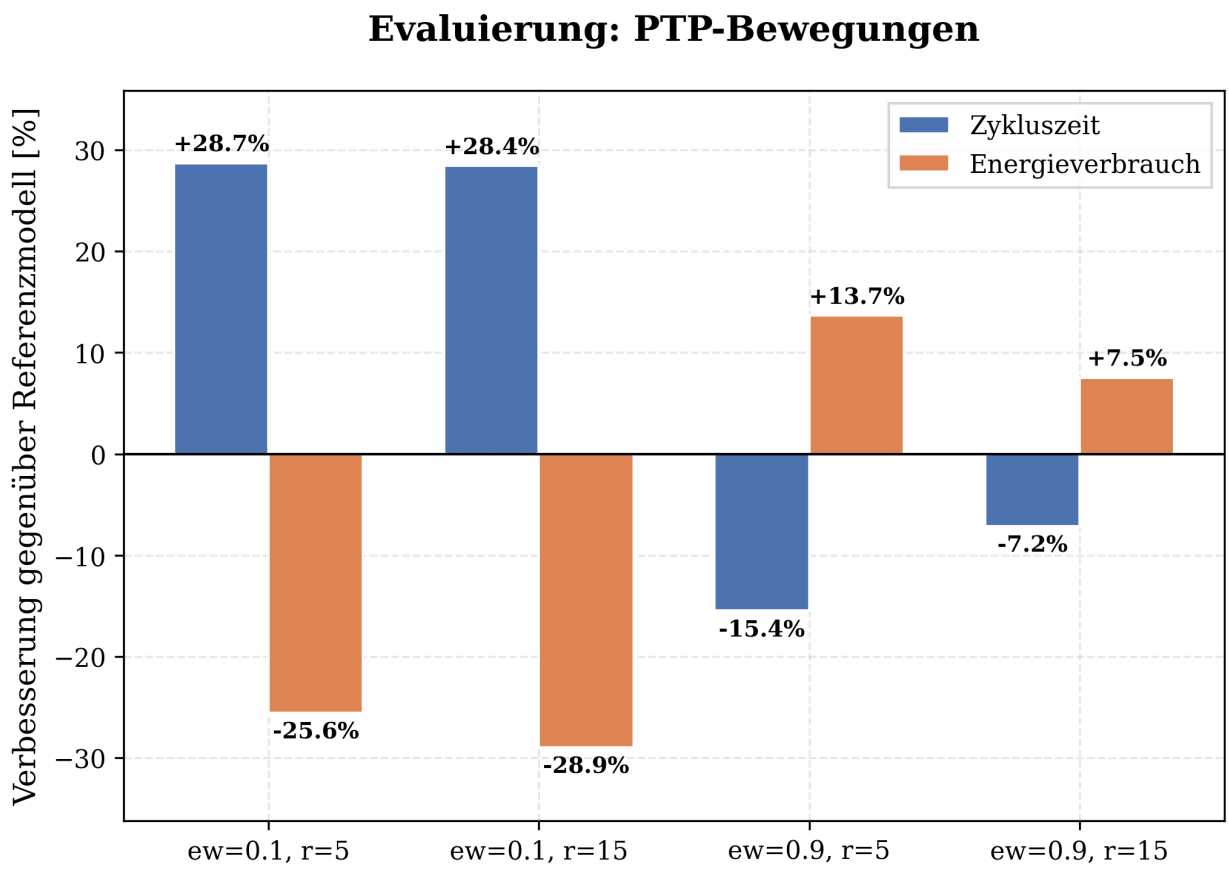


Abbildung 7.3: Durchschnittliche Taktzeit- und Energieverbesserung der PTP-Bewegungen im unbeladenen Zustand: Die Abbildung zeigt die durchschnittliche Verbesserung bei verschiedenen Gewichtungsfaktoren (ew) und verschiedenen Variationsradien (r).

Die Ergebnisse zeigen, dass der Agent den geforderten Kompromiss zwischen Taktzeit und Energieverbrauch für PTP-Bewegungen klar erlernt hat. Positive Prozentwerte bezeichnen dabei stets eine Verbesserung gegenüber dem Referenzmodell. Eine Taktzeit-Verbesserung von 10 % bedeutet demnach eine um 10 % schnellere Bewegungsausführung, eine Verbesserung des Energieverbrauchs von 10 % entspricht einem um 10 % geringeren Energiebedarf pro Bewegung. Negative Prozentwerte beschreiben analog eine Verschlechterung gegenüber dem Referenzmodell, also eine langsamere Ausführung beziehungsweise einen höheren Energieverbrauch. Bei `energy_weight` $= 0.1$ erzielt er eine durchschnittliche Taktzeit-Verbesserung von rund 29 % gegenüber dem Referenzmodell, nimmt dafür jedoch einen erhöhten Energieverbrauch von rund 26 % bis 29 % in Kauf. Bei `energy_weight` $= 0.9$ kehrt sich das Verhalten erwartungsgemäß um. Der Agent reduziert den Energieverbrauch um rund 8 % bis 14 %, akzeptiert dabei aber eine längere Zykluszeit von rund 7 % bis 15 %. Damit erfüllt das Modell die Evaluierungskriterien K3.1.1 und K3.1.2. Besonders hervorzuheben ist die stabile Generalisierung bei `energy_weight` $= 0.1$: Die Ergebnisse bei $r = 5\,\text{cm}$ und $r = 15\,\text{cm}$ sind nahezu identisch. Bei `energy_weight` $= 0.9$ ist hingegen eine stärkere Abhängigkeit vom Variationsradius zu beobachten. Die Energie-Verbesserung reduziert sich von 13.7 % bei $r = 5\,\text{cm}$ auf 7.5 % bei $r = 15\,\text{cm}$. Dies lässt sich darauf zurückführen, dass die Energieoptimierung eine deutlich komplexere Strategie erfordert als die reine Taktzeitminimierung. Während eine Reduktion der Taktzeit primär durch die Wahl maximaler Bewegungsparameter erreicht wird, weitgehend unabhängig von den spezifischen kinematischen Eigenschaften der Trajektorie, wird die Energieeffizienz von einer Vielzahl weiterer Faktoren beeinflusst. Hierzu zählen unter anderem die Gelenkkonfigurationen sowie die Taktzeit selbst, da eine zu langsame Bewegung zwar die dynamische Last senkt, jedoch den Energiebedarf für das Halten der Pose über den längeren Zeitraum erhöht. Da die kinematischen Eigenschaften bei einem größeren Variationsradius stärker variieren, kann die erlernte Strategie des Modells hier nicht mehr dasselbe Optimierungspotenzial ausschöpfen wie bei geringeren Variationsradien.

7.3.2 PTP-Bewegungen bei beladenem Zustand

Da die Generalisierungsfähigkeit des Agenten bereits im vorherigen Abschnitt nachgewiesen wurde, wird hier ausschließlich die mittlere Generalisierungsstufe von $r = 10\,\text{cm}$ betrachtet. Untersucht werden zwei Grenzwerte des Beschleunigungsbudgets: $a_{\text{max,zul}} = 13\,\text{m/s}^2$ als untere praxisrelevante Grenze sowie $a_{\text{max,zul}} = 18\,\text{m/s}^2$ als repräsentativer mittlerer Wert mit mehr Spielraum für den Agenten.

Abbildung 7.4 zeigt die Ergebnisse. Der grundlegende Kompromiss zwischen Taktzeit und Energie bleibt auch im beladenen Zustand klar erhalten. Bei `energy_weight` $= 0.1$ erzielt der Agent Taktzeit-Verbesserungen von 23.2 % bzw. 27.4 %, bei `energy_weight` $= 0.9$ Energie- Verbesserungen von 13.6 % bzw. 12.9 %. Der Agent reagiert damit auch unter Beladung korrekt auf den Gewichtungsfaktor.

Besonders aufschlussreich ist der Vergleich zwischen den beiden Beschleunigungsbudgets, da hier deutlich wird, wie der Agent den erweiterten Handlungsspielraum zur Optimierung der Zielgrößen nutzt. Bei `energy_weight` $= 0.1$ profitiert der Agent vom höheren Budget. Die Taktzeit-Verbesserung steigt von 23.2 % auf 27.4 %, da der Agent bei $a_{\text{max,zul}} = 18\,\text{m/s}^2$ höhere Beschleunigungen wählen kann ohne das Sicherheitsbudget zu verletzen. Bei `energy_weight` $= 0.9$ hingegen sind die Ergebnisse mit 13.6 % bzw. 12.9 % verbessertem Energieverbrauch nahezu identisch. Dies bestätigt die innere Logik des Agenten. Um Energie zu sparen wählt er ohnehin niedrige Beschleunigungen, sodass das die maximal zulässige Beschleunigung in diesem Modus nicht der limitierende Faktor ist und ein höherer Grenzwert keinen zusätzlichen Nutzen bringt.

Im Vergleich zum unbeladenen Fall fällt die Taktzeit-Verbesserung bei `energy_weight` $= 0.1$ etwas

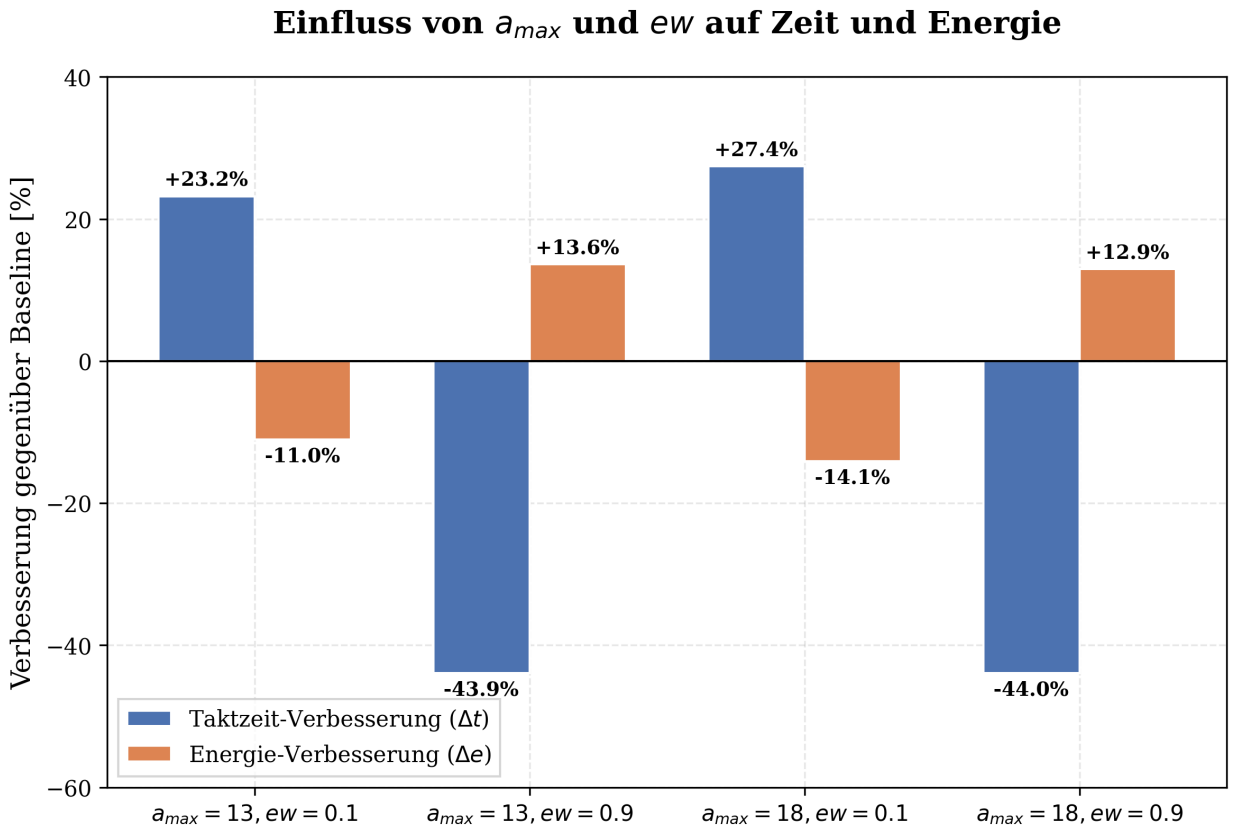


Abbildung 7.4: Durchschnittliche Taktzeit- und Energieverbesserung der PTP-Bewegungen im beladenen Zustand: Die Abbildung zeigt den Einfluss von $a_{\text{max,zul}}$ und `energy_weight` (ew) auf Taktzeit- und Energieverbesserung gegenüber dem Referenzmodell für PTP-Bewegungen im beladenen Zustand bei einem Variationsradius von $r = 10\,\text{cm}$.

geringer aus (23.2 % statt 28.7 % bei $a_{\text{max,zul}} = 13\,\text{m/s}^2$), was auf das durch die Beladung eingeschränkte Beschleunigungsbudget zurückzuführen ist. Der Agent erkennt die Beladung korrekt über den Beobachtungsvektor und wählt entsprechend konservativere Parameter, um die Sicherheitsanforderung A2.3 einzuhalten.

7.3.3 LIN-Bewegungen im unbeladenen Zustand

Nachfolgend wird das Optimierungsverhalten des Agenten für LIN-Bewegungen im unbeladenen Zustand untersucht, um die erlernte Strategie isoliert von den zusätzlichen Sicherheitsanforderungen durch Beladung beurteilen zu können.

Abbildung 7.5 zeigt die entsprechenden Ergebnisse für LIN-Bewegungen. Das Optimierungsverhalten folgt prinzipiell demselben Muster wie bei PTP-Bewegungen, ist jedoch deutlich schwächer ausgeprägt. Dies ist direkt auf die in Abschnitt 5.3.3 beschriebene *Reward*-Struktur zurückzuführen: Da die Optimierungsanteile bei LIN-Bewegungen mit einem festen Faktor von 0.2 gewichtet werden, erhält der Agent einen deutlich schwächeren Lernimpuls zur Optimierung von Taktzeit und Energie. Die erlernte Strategie ist entsprechend konservativer und die erzielten Verbesserungen fallen insgesamt geringer aus als im PTP-Fall.

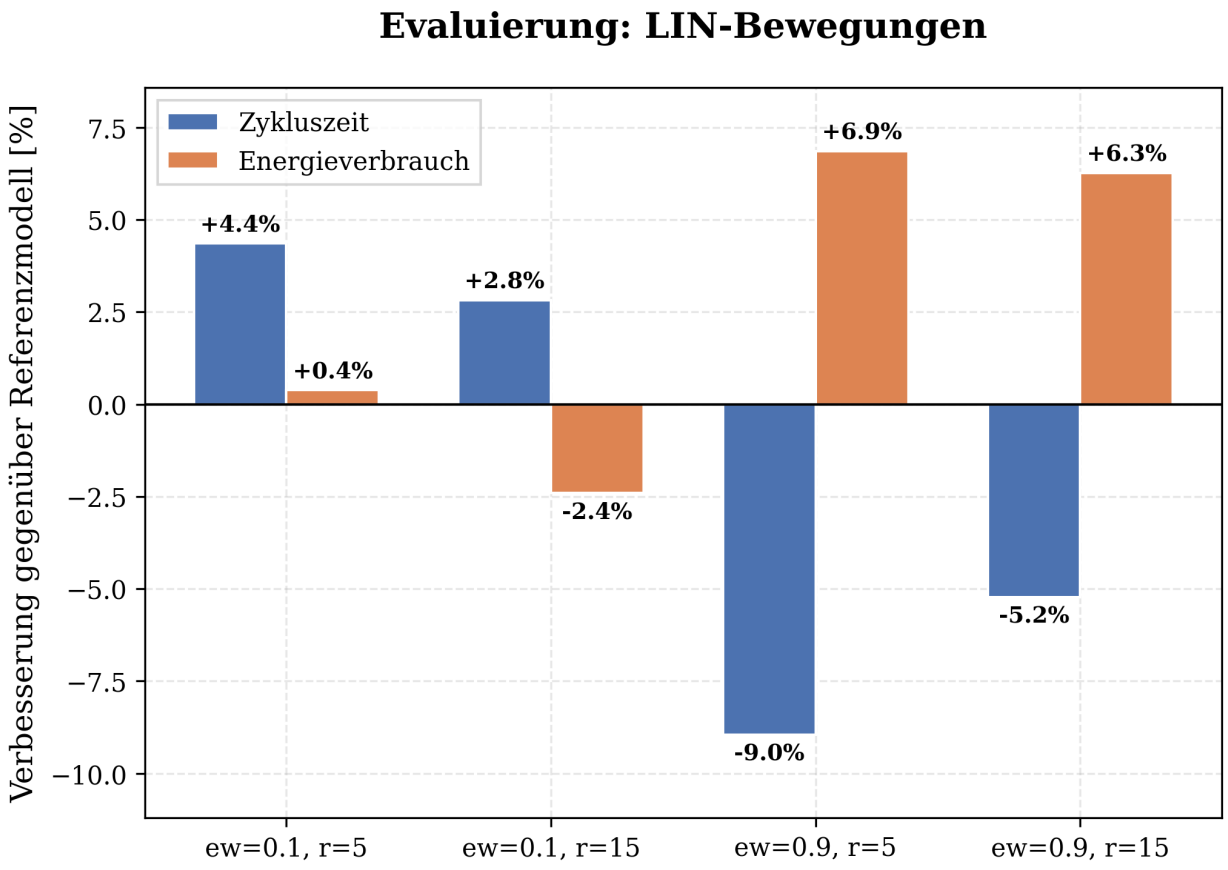


Abbildung 7.5: Durchschnittliche Taktzeit- und Energieverbesserung gegenüber dem Referenzmodell für LIN-Bewegungen im unbeladenen Zustand: Die Abbildung zeigt die durchschnittliche Verbesserung bei verschiedenen Gewichtungsfaktoren (ew) und Variationsradien (r).

Bei `energy_weight` $= 0.1$ erzielt der Agent positive Taktzeit-Verbesserungen von 4.4 % bei $r = 5\,\text{cm}$ bzw. 2.8 % bei $r = 15\,\text{cm}$, bei gleichzeitig neutralem bis leicht negativem Energieverbrauch

von +0.4 % bzw. −2.4 %. Bei `energy_weight` = 0.9 erzielt der Agent Energieverbesserungen von 6.9 % bei $r = 5\,\text{cm}$ bzw. 6.3 % bei $r = 15\,\text{cm}$, nimmt dafür jedoch Taktzeit-Verschlechterungen von −9.0 % bzw. −5.2 % in Kauf. Der Gewichtungsfaktor `energy_weight` hat damit auch bei LIN-Bewegungen einen erkennbaren Einfluss auf die Parameterwahl des Agenten, wenngleich dieser schwächer ausgeprägt ist als im PTP-Fall.

Auffällig ist dabei, dass `energy_weight` bei LIN-Bewegungen nicht direkt in der *Reward*-Funktion verankert ist. Der Optimierungsanteil wird dort unabhängig vom Gewichtungsfaktor stets mit dem festen Faktor 0.2 skaliert. Der Agent hat dennoch gelernt, `energy_weight` als relevantes Zustandsmerkmal zu interpretieren und seine Parameterwahl entsprechend anzupassen. Da `energy_weight` Teil des Beobachtungsvektors ist, lernt der Agent während des Trainings implizit, dieses Merkmal auch für LIN-Bewegungen zu berücksichtigen, obwohl der direkte *Reward*-Anreiz hierfür fehlt.

7.3.4 LIN-Bewegungen im beladenen Zustand

Analog zu Abschnitt 7.3.2 wird auch für LIN-Bewegungen der Einfluss des Beschleunigungsbudgets $a_{\text{max,zul}}$ auf das Optimierungsverhalten des Agenten im beladenen Zustand untersucht. Betrachtet werden die Werte $a_{\text{max,zul}} = 13\,\text{m/s}^2$ und $a_{\text{max,zul}} = 18\,\text{m/s}^2$ bei einem Variationsradius von $r = 10\,\text{cm}$. Dabei repräsentiert $a_{\text{max,zul}} = 13\,\text{m/s}^2$ die untere praxisrelevante Grenze des Beschleunigungsbudgets, bei der der Agent nur wenig Spielraum für aggressive Parameterwahl hat, während $a_{\text{max,zul}} = 18\,\text{m/s}^2$ einen Wert darstellt, der dem Agenten deutlich mehr Freiraum bei der Wahl der Dynamikparameter lässt.

Abbildung 7.6 zeigt die Ergebnisse. Es lassen sich zwei klar unterschiedliche Strategien erkennen, die stark vom verfügbaren Beschleunigungsbudget abhängen.

Bei $a_{\text{max,zul}} = 13\,\text{m/s}^2$ zeigt der Agent extreme Taktzeit-Verschlechterungen von −86.3 % bei einem Gewichtungsfaktor von 0.1 beziehungsweise −148.1 % bei einem Gewichtungsfaktor von 0.9. Das Zusammenspiel aus Beladung und engem Beschleunigungsbudget zwingt den Agenten zu sehr konservativen Parametern. Auffällig ist dabei das Verhalten bei `energy_weight` = 0.9: Trotz Energiefokus erzielt der Agent keine Energieverbesserung, sondern eine Verschlechterung von −16.5 %. Dies lässt sich darauf zurückführen, dass die massive Taktzeitverlängerung den gegenteiligen Effekt auf den Energieverbrauch haben kann, ein sehr langsam fahrender Roboter hält Gelenkposen über einen längeren Zeitraum und erhöht damit den statischen Energiebedarf, was die durch niedrige Dynamik eingesparte Bewegungsenergie überkompensiert.

Bei $a_{\text{max,zul}} = 18\,\text{m/s}^2$ reduzieren sich die Taktzeit-Verschlechterungen auf −9.8 % bei `energy_weight` = 0.1 beziehungsweise −9.7 % bei `energy_weight` = 0.9, während die Energieverbesserung mit +1.3 % bzw. +1.1 % leicht positiv ausfällt. Mit ausreichend Freiraum im Beschleunigungsbudget stabilisiert sich das Verhalten des Agenten auf einem moderaten Niveau. Dabei ist auffällig, dass die Ergebnisse für beide Gewichtungsfaktoren nahezu identisch sind. Dies deutet darauf hin, dass der Agent bei beladenen LIN-Bewegungen eine feste Sicherheitsstrategie erlernt hat, die weitgehend unabhängig von `energy_weight` greift. Dieses Verhalten ist konsistent mit der *Reward*-Struktur: Der Beladungszustand `is_loaded` ist Teil des Beobachtungsvektors, und der Agent hat gelernt, für beladene LIN-Bewegungen die Sicherheitsanforderung gegenüber der Optimierung stark zu priorisieren.

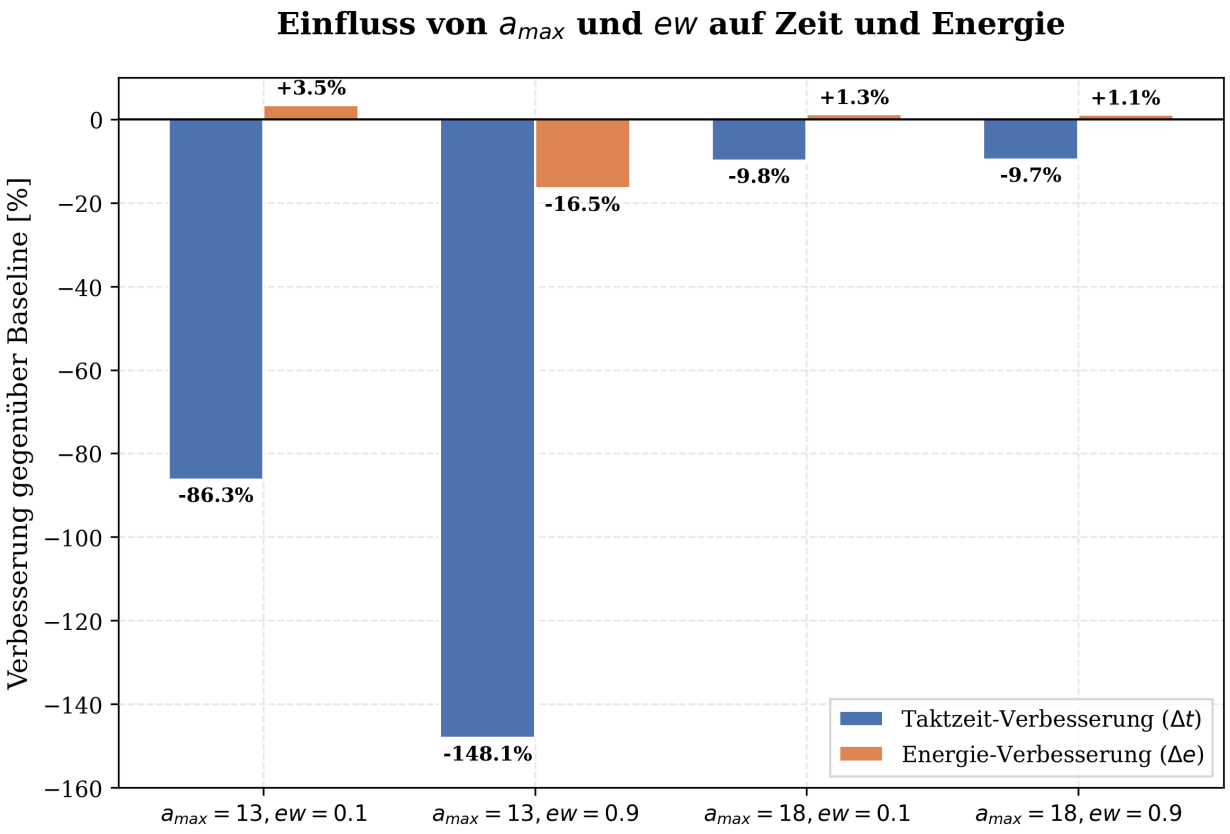


Abbildung 7.6: Durchschnittliche Taktzeit- und Energieverbesserung der LIN-Bewegungen im beladenen Zustand: Die Abbildung zeigt den Einfluss von $a_{\text{max,zul}}$ und `energy_weight` (ew) auf Taktzeit- und Energieverbesserung gegenüber dem Referenzmodell für LIN-Bewegungen im beladenen Zustand bei einem Variationsradius von $r = 10\,\text{cm}$.

7.3.5 Bewertung

Für PTP-Bewegungen erfüllt das Modell die Anforderung A3.1 sowohl im unbeladenen als auch im beladenen Zustand. Hier hat es den Kompromiss zwischen Taktzeit und Energieverbrauch in Abhängigkeit von `energy_weight` zuverlässig erlernt und erzielt signifikante Verbesserungen gegenüber dem Referenzmodell. Gleiches gilt für unbeladene LIN-Bewegungen, bei denen der Agent trotz konservativer Grundstrategie messbare Verbesserungen erzielt. Die Einschränkungen des Systems beschränken sich damit auf einen klar eingegrenzten Fall: Beladene LIN-Bewegungen an der unteren Grenze des Beschleunigungsbudgets.

Insbesondere bei $a_{\text{max,zul}} = 13\,\text{m/s}^2$ wählt der Agent Parameter, die zu erheblichen Taktzeit-Verschlechterungen von bis zu $-148.1\,\%$ führen. Diese sind jedoch keine Fehlfunktion, sondern eine direkte Konsequenz der *Reward*-Struktur, die Sicherheit gegenüber Optimalität priorisiert, und entsprechen damit der in Anforderung A2.3 definierten Priorität. Zu berücksichtigen ist, dass diese Verschlechterungen die in der Gesamtaufgabe erzielten Optimierungen teilweise negieren können. Da jedoch nicht alle sondern nur beladene LIN-Bewegungen betroffen sind und LIN-Bewegungen typischerweise nur für kurze Greif- und Ablegevorgänge eingesetzt werden, entfällt der überwiegende Anteil der Ausführungszeit auf PTP-Bewegungen. Der praktische Einfluss auf die Gesamtzykluszeit ist damit deutlich geringer als die isolierten Werte vermuten lassen. Für Anwendungen mit besonders engen Taktzeit-Anforderungen empfiehlt sich eine Anpassung des Beschleunigungsbudgets oder eine gezielte Erweiterung der *Reward*-Gewichtung für beladene LIN-Bewegungen.

7.4 Anforderungserfüllung

Die Übersicht in Tabelle 7.1 fasst die Ergebnisse der Evaluation aus den Abschnitten 7.2 und 7.3 zusammen und bewertet die Erfüllung der in Kapitel 4 definierten Anforderungen anhand der messbaren Evaluierungskriterien.

Tabelle 7.1: Übersicht der Anforderungserfüllung

Kriterium	Anf.	Ergebnis	Bewertung
K1.1.1	A1.1	Der Agent bestimmt `max_vel` und `max_acc` prädiktiv vor Bewegungsausführung. Die Parameterübergabe ist im Ausführungslog nachvollziehbar.	Vollständig erfüllt
K2.1.1	A2.1	In allen Testszenarien kam es zu keiner Ablehnung von Bewegungszielen durch MoveIt aufgrund der Verletzungen von Robotergrenzen.	Vollständig erfüllt
K2.3.1	A2.3	Erfolgsraten von 96 % bis 100 % über alle getesteten Bewegungen bei $a_{\text{max,zul}} = 13\,\text{m/s}^2$. Vereinzelte Schlupf-Ereignisse treten ausschließlich maximalem Variationsradius ($r = 15\,\text{cm}$) auf.	Erfüllt mit Einschränkung
K3.1.1	A3.1	Bei `energy_weight` $= 0.1$ erzielt der Agent eine durchschnittliche Taktzeit-Verbesserung von rund 29 % gegenüber dem Referenzmodell für PTP-Bewegungen. Für beladene LIN-Bewegungen bei $a_{\text{max,zul}} = 13\,\text{m/s}^2$ treten hingegen erhebliche Taktzeit-Verschlechterungen auf.	Erfüllt mit Einschränkung
K3.1.2	A3.1	Bei `energy_weight` $= 0.9$ erzielt der Agent eine durchschnittliche Energie-Verbesserung von rund 10 % bis 14 % gegenüber dem Referenzmodell für PTP-Bewegungen. Für beladene LIN-Bewegungen bei $a_{\text{max,zul}} = 13\,\text{m/s}^2$ wird hingegen keine Energieverbesserung erzielt.	Erfüllt mit Einschränkung
K4.1.1	A4.1	Der Agent erfüllt A2 und erzielt messbare Verbesserungen gemäß A3 für alle getesteten unbekannten Bewegungen. Bei $r = 10\,\text{cm}$ werden 99 % Erfolgsrate erreicht, bei $r = 15\,\text{cm}$ 96 %.	Erfüllt mit Einschränkung

Funktionalität (A1) Anforderung A1.1 ist vollständig erfüllt. Der Agent bestimmt die Dynamikparameter `max_vel` und `max_acc` ausschließlich auf Basis statischer Eingangsgrößen, bevor die Bewegung ausgeführt wird. Dies adressiert direkt die in Kapitel 3 identifizierte Forschungslücke der fehlenden vorgelagerten, prädiktiven Parametrierung.

Sicherheit (A2) Anforderung A2.1 ist vollständig erfüllt, da es in keinem Testszenario zu Ausführungsfehlern kam. Anforderung A2.3 ist mit einer Einschränkung erfüllt: Für den Bereich $a_{\text{max,zul}} = 13\,\text{m/s}^2$ werden Erfolgsraten von 96 % bis 100 % erreicht. Die verbleibenden Schlupf-Ereignisse treten dabei ausschließlich bei maximalem Variationsradius auf und können durch einen konservativen Sicherheitsfaktor bei der Vorberechnung von $a_{\text{max,zul}}$ weiter reduziert werden. Da das System simulationsbasiert arbeitet, können neue Bewegungen zudem vor dem realen Einsatz in der Si-

mulation validiert werden, sodass potenzielle Schlupf-Ereignisse frühzeitig erkannt und korrigiert werden können, bevor die Parametrierung auf das reale System übertragen wird. Die verbleibenden Schlupf-Ereignisse sind daher als nicht kritisch einzustufen.

Optimalität (A3) Anforderung A3.1 ist für PTP-Bewegungen sowie unbeladene LIN-Bewegungen vollständig erfüllt. Der Agent hat den Kompromiss zwischen Taktzeit und Energieverbrauch in Abhängigkeit von `energy_weight` zuverlässig erlernt und erzielt klare Verbesserungen gegenüber dem Referenzmodell. Für beladene LIN-Bewegungen bei engem Beschleunigungsbudget ($a_{\mathrm{max,zul}} = 13\,\mathrm{m/s^2}$) ist die Anforderung mit einer Einschränkung erfüllt. Der Agent priorisiert in diesem Fall konsequent Sicherheit gegenüber Optimalität, was zu erheblichen Taktzeit-Verschlechterungen führt. Dies ist eine direkte Konsequenz der *Reward*-Struktur und entspricht der definierten Priorität von A2.3 gegenüber A3.1.

Adaptivität (A4) Anforderung A4.1 ist mit einer Einschränkung erfüllt. Der Agent generalisiert zuverlässig auf strukturell ähnliche, nicht im Training enthaltene Bewegungen innerhalb der Roboterstation. Für den Variationsradius von $r = 10\,\mathrm{cm}$ werden 99 % Erfolgsrate erreicht. Bei $r = 15\,\mathrm{cm}$ nimmt die Erfolgsrate leicht auf 96 % ab, was bedeutet dass bei grundlegend neuen Bewegungen im Extremfall ein Feintuning des Modells empfohlen wird.

Die Ergebnisse zeigen damit, dass das entwickelte System das übergeordnete Ziel der Arbeit erreicht und die manuelle Parametrierarbeit in der industriellen Robotik deutlich reduzieren kann. Das Modell kann auf Stationsebene eingesetzt werden, um sichere und weitgehend optimierte Bewegungen automatisiert zu parametrieren, wobei die Einschränkungen sich auf beladene LIN-Bewegungen bei engem Beschleunigungsbudget beschränken. Für den Bereich von $r \leq 10\,\mathrm{cm}$, der typische Variationen innerhalb einer Roboterstation des verwendeten Roboters abdeckt, parametriert der Agent nahezu jede neue Bewegung korrekt und ohne manuellen Eingriff. Selbst im Extremfall $r = 15\,\mathrm{cm}$ erfordert lediglich jede zwanzigste neue Bewegung eine manuelle Nachparametrierung. Gegenüber einer vollständig manuellen Vorgehensweise, bei der jede Bewegung einzeln abgestimmt werden muss, bedeutet dies eine erhebliche Reduktion des Parametrieraufwands. Dabei ist klar abzugrenzen, dass die vom Agenten gewählten Parameter nicht die theoretisch maximale Optimierung hinsichtlich Taktzeit oder Energieverbrauch erreichen. Dies ist jedoch eine bewusste Konsequenz der Priorisierung von Sicherheit in der *Reward* -Funktion. Der Agent wählt Parameter die zuverlässig sicher und prozessstabil sind, und optimiert innerhalb dieses Rahmens. Das primäre Ziel, die Reduktion des manuellen Parametrieraufwands, wird damit klar erfüllt.

7.5 Ausblick

Die vorliegende Arbeit hat gezeigt, dass ein RL-Agent auf Stationsebene in der Lage ist, sichere und optimierte Dynamikparameter für strukturell ähnliche Bewegungen automatisiert abzuleiten. Eine naheliegende Erweiterung ist die Verallgemeinerung dieses Ansatzes über die Stationsebene hinaus, hin zu einem Modell das für beliebige Roboterbewegungen einsetzbar ist.

Die Grundlage hierfür wurde in der vorliegenden Arbeit bereits bewusst gelegt. Der Beobachtungsraum verwendet keine absoluten Gelenkwinkel oder kartesischen Posen, sondern kinematische Merkmale wie Gelenkdistanz, kartesische Distanz und Höhenänderung. Dadurch erzeugen strukturell ähnliche Bewegungen ähnliche Beobachtungsvektoren, was die Generalisierung prinzipiell über Stationsgrenzen hinaus ermöglicht. Um dieses Potenzial auszuschöpfen, müsste die Trainingsbibliothek erheblich erweitert werden, sodass sie den gesamten kinematischen Merkmalsraum möglicher

Roboterbewegungen repräsentativ abdeckt.

Dies bringt jedoch eine Reihe von Folgeproblemen mit sich. Zunächst ist die systematische Erzeugung einer ausreichend großen und gleichmäßig verteilten Bewegungsbibliothek mit erheblichem Aufwand verbunden, da sichergestellt werden muss, dass alle relevanten kinematischen Eigenschaften abgedeckt sind. Darüber hinaus bricht eine vollständige Erzeugung der Trainingsbewegungen den in dieser Arbeit verwendeten Vergleich mit einem Referenzmodell, da für jede erzeugte Bewegung kein vorberechneter Referenzwert vorliegt. Eine mögliche Lösung wäre, für jede erzeugte Bewegung zunächst einen Referenzlauf mit festen Parametern durchzuführen und diesen als Normierungsbasis zu verwenden. Dies würde jedoch die Anzahl der benötigten Simulationsschritte pro Episode verdoppeln und damit die Trainingszeit erheblich erhöhen.

Um dies zu kompensieren, wäre eine Parallelisierung des Trainings durch die simultane Simulation mehrerer Roboter erforderlich, wie sie beispielsweise in modernen Physik-Engines wie *Isaac Sim* prinzipiell möglich ist. Erst die Kombination aus systematisch erzeugten Trainingsbewegungen, einem angepassten Normierungskonzept und einer parallelisierten Simulationsumgebung würde eine Generalisierung auf stationsübergreifende Bewegungen realistisch ermöglichen.

Unabhängig von der Generalisierungsebene bietet auch der vorliegende stationsbezogene Ansatz noch weiteres Entwicklungspotenzial. So wurde die Robustheit des Agenten gegenüber Messrauschen in den Eingangsparametern nicht evaluiert und bleibt weiterführenden Untersuchungen vorbehalten. Darüber hinaus wäre die Übertragung des trainierten Modells auf einen realen Roboter ein wichtiger nächster Schritt, um die *Sim-to-Real*-Übertragbarkeit des Ansatzes zu validieren und damit den Weg von der simulationsbasierten Entwicklung hin zum praxistauglichen Einsatz in der industriellen Fertigung zu ebnen.

Literaturverzeichnis

Akbari, A., Mozaffari, S., Singh, R., Ahmadi, M., & Alirezaee, S., (2023). Movement Optimization of Robotic Arms for Energy and Time Reduction using Evolutionary Algorithms. *2023 International Symposium on Signals, Circuits and Systems (ISSCS)*, 1–4.

Azeez, M. I., Abdelhaleem, A. M. M., Elnaggar, S., Moustafa, K. A. F., & Atia, K. R., (2023). Optimization of PID Trajectory Tracking Controller for a 3-DOF Robotic Manipulator Using Enhanced Artificial Bee Colony Algorithm. *Scientific Reports*, *13*(1), 11164. https://doi.org/10.1038/s41598-023-37895-3

Bobrow, J., Dubowsky, S., & Gibson, J., (1985). Time-Optimal Control of Robotic Manipulators Along Specified Paths. *The International Journal of Robotics Research*, *4*(3), 3–17. https://doi.org/10.1177/027836498500400301

Burggräf, P., Steinberg, F., Heinbach, B., & Bamberg, M., (2022). Reinforcement Learning for Process Time Optimization in an Assembly Process Utilizing an Industry 4.0 Demonstration Cell [Leading manufacturing systems transformation – Proceedings of the 55th CIRP Conference on Manufacturing Systems 2022]. *Procedia CIRP*, *107*, 1095–1100. https://doi.org/https://doi.org/10.1016/j.procir.2022.05.114

Chen, T., Yang, W., Li, S., & Luo, X., (2025). Data-Driven Calibration of Industrial Robots: A Comprehensive Survey. *IEEE/CAA Journal of Automatica Sinica*, *12*(8), 1544–1567. https://doi.org/10.1109/JAS.2025.125237

Gasparetto, A., & Zanotto, V., (2007). A new method for smooth trajectory planning of robot manipulators. *Mechanism and Machine Theory*, *42*(4), 455–471. https://doi.org/https://doi.org/10.1016/j.mechmachtheory.2006.04.002

Gasparetto, A., & Zanotto, V., (2008). A technique for time-jerk optimal planning of robot trajectories. *Robotics and Computer-Integrated Manufacturing*, *24*(3), 415–426. https://doi.org/https://doi.org/10.1016/j.rcim.2007.04.001

Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S., (2018). Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. https://arxiv.org/abs/1801.01290

Heuss, L., Gonnermann, C., & Reinhart, G., (2022). An extendable framework for intelligent and easily configurable skills-based industrial robot applications. *The International Journal of Advanced Manufacturing Technology*, *120*(9), 6269–6285. https://doi.org/10.1007/s00170-022-09071-w

Hovgard, M., Lennartson, B., & Bengtsson, K., (2021). Applied energy optimization of multi-robot systems through motion parameter tuning. *CIRP Journal of Manufacturing Science and Technology*, *35*, 422–430. https://doi.org/https://doi.org/10.1016/j.cirpj.2021.07.012

Kober, J., Bagnell, J. A., & Peters, J., (2013). Reinforcement learning in robotics: A survey. *The International Journal of Robotics Research*, *32*(11), 1238–1274. https://doi.org/10.1177/0278364913495721

Lobbezoo, A., Qian, Y., & Kwon, H.-J., (2021). Reinforcement Learning for Pick and Place Operations in Robotics: A Survey. *Robotics*, *10*(3). https://doi.org/10.3390/robotics10030105

Luo, L.-P., Yuan, C., Yan, R.-J., Yuan, Q., Wu, J., Shin, K.-S., & Han, C.-S., (2015). Trajectory Planning for Energy Minimization of Industry Robotic Manipulators Using the Lagrange Interpolation Method. *International Journal of Precision Engineering and Manufacturing*, *16*(5), 911–917. https://doi.org/10.1007/s12541-015-0119-9

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D., (2015). Human-level control through deep reinforcement learning. *Nature*, *518*(7540), 529–533. https://doi.org/10.1038/nature14236

Nocedal, J., & Wright, S. J., (2006). *Numerical optimization*. Springer.

NVIDIA Corporation, (2023). Isaac Sim [Abgerufen am: 29.03.2026].

Pan, J., Chen, F., Han, D., Ke, S., Wei, Z., & Ding, H., (2025). Adaptive process parameters decision-making in robotic grinding based on meta-reinforcement learning. *Journal of Manufacturing Processes*, *137*, 376–396. https://doi.org/https://doi.org/10.1016/j.jmapro.2025.02.005

Paranjape, A., Quader, N., Uhlmann, L., Berkels, B., Wolfschläger, D., Schmitt, R. H., & Bergs, T., (2025). Reinforcement Learning Agent for Multi-Objective Online Process Parameter Optimization of Manufacturing Processes. *Applied Sciences*, *15*(13). https://doi.org/10.3390/app15137279

PickNik Robotics, (2021). MoveIt [Abgerufen am: 29.03.2026].

Pilz GmbH & Co. KG, (2024). *Pilz Industrial Motion Planner*. Verfügbar 18. März 2026 unter https://moveit.picknik.ai/main/doc/how_to_guides/pilz_industrial_motion_planner/pilz_industrial_motion_planner.html

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O., (2017). Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*.

Sheng, L., & Li, W., (2018). Optimization Design by Genetic Algorithm Controller for Trajectory Control of a 3-RRR Parallel Robot. *Algorithms*, *11*(1). https://doi.org/10.3390/a11010007

Sutton, R. S., & Barto, A. G., (2018). *Reinforcement Learning: An Introduction* (2. Aufl.). MIT Press. http://incompleteideas.net/book/the-book.html

Talbi, E.-G., (2009). *Metaheuristics: from design to implementation*. John Wiley & Sons.

Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U., De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG, A., et al., (2024). Gymnasium: A Standard Interface for Reinforcement Learning Environments. *arXiv preprint arXiv:2407.17032*.

Xu, M., & Wang, J., (2023). Deep Reinforcement Learning for Parameter Tuning of Robot Visual Servoing. *ACM Trans. Intell. Syst. Technol.*, *14*(2). https://doi.org/10.1145/3579829

Erklärung zum Einsatz von KI-basierten Tools

KI-basiertes Tool	Letzter Zugriff	Anwendungsfall	Umfang	Anmerkungen
Claude Sonnet 4.6	18.03.2026	Textüberarbeitung und Paraphrasierung	Gesamte Arbeit	-
Gemini 3 Flash Preview	18.03.2026	Textüberarbeitung und Paraphrasierung	Gesamte Arbeit	-

Selbstständigkeitserklärung

Ich versichere hiermit, dass ich die von mir eingereichte Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.



Garching, 20. April 2026 (Unterschrift)